CSE 385: Analysis of Algorithms (Honors)

Rezaul A. Chowdhury
Department of Computer Science
SUNY Stony Brook
Spring 2026

"Theory is when you know everything, but nothing works.
Practice is when everything works but no one knows why.
In our lab, theory and practice are combined:
nothing works and no one knows why."

— A practical theoretician

Basic Logistics: Who/Where/When

- **Lecture Time:** MW 6:30 pm – 7:50 pm (lecture)
  M 5:30 pm – 6:25 pm (~~recitation~~, lecture)
- **Location:** NCS 120, West Campus
  Lectures may be live streamed via Zoom

- **Instructor:** Rezaul Chowdhury
- **Office Hours:** MW 11 am - 12 pm
  Zoom link available on Brightspace
- **Email:** rezaul@cs.stonybrook.edu

- **TA:** Pritom Kundu (prkundu@cs.stonybrook.edu)
- **TA Office Hours:** TBA

- **Class Webpage:**
  https://www3.cs.stonybrook.edu/~rezaul/CSE385-S26.html
- **Piazza:** https://piazza.com/stonybrook/spring2026/cse385

Topics to be Covered

The following topics will be covered (hopefully)

- elementary data structure
- sorting and searching
- recurrence relations and divide-and-conquer algorithms
- dynamic programming
- graph algorithms
- randomized algorithms
- parallel algorithms and multithreaded computations
- NP-completeness and approximation algorithms
- FFT (Fast Fourier Transforms)

Warning: We will use Math

This is **Analysis** **of Algorithms**, not just **Algorithms**.

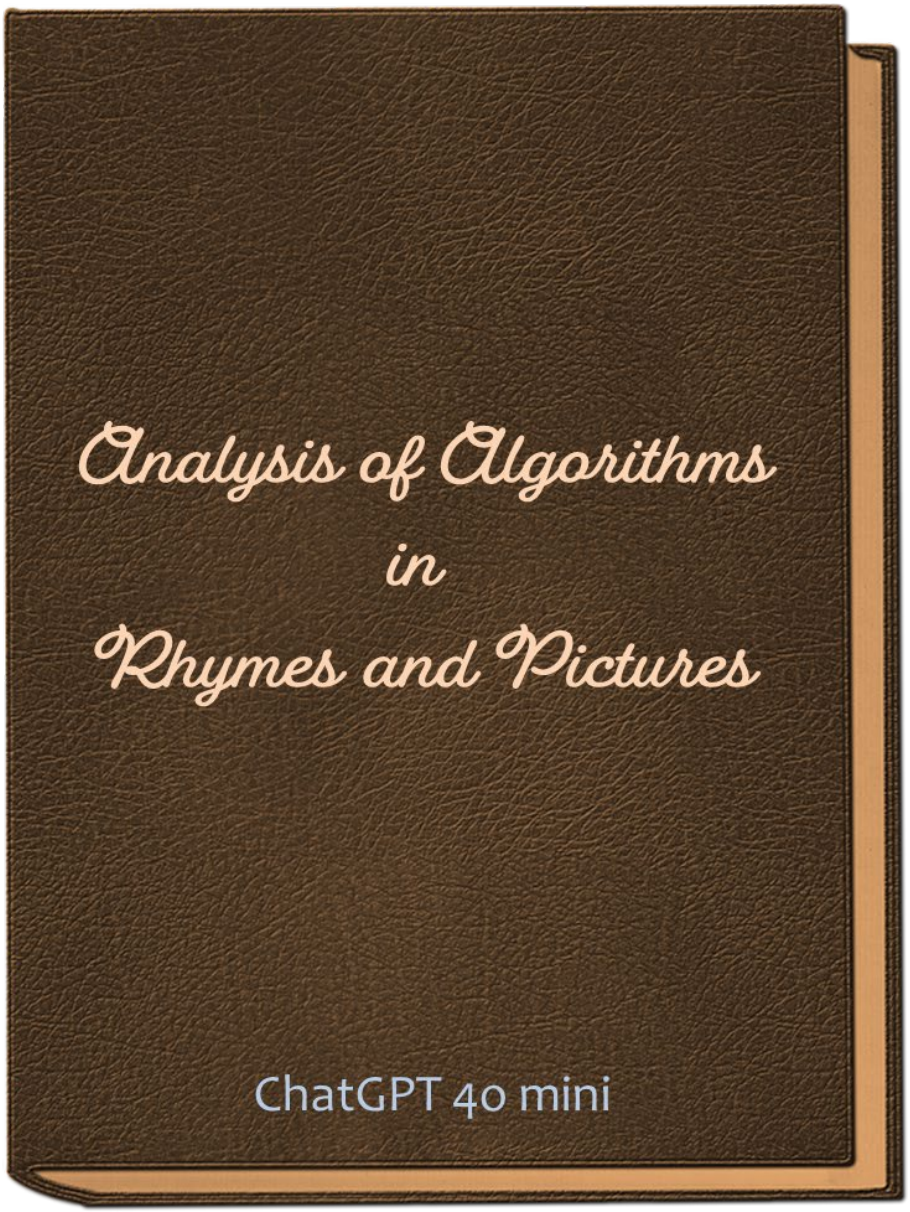
Official Course Description of CSE 385 (SBU Course Catalog):

Algorithmic **design** and **analysis** for Computer Science Honors students. **Mathematical analysis** of a variety of computer algorithms including searching, sorting, matrix multiplication, fast Fourier transform, and graph algorithms. Time and space complexity. Upper-bound, lower-bound, and average-case analysis. Randomization. Introduction to NP completeness. Some machine computation is required for the implementation and comparison of algorithms.

We will use **math** to do **mathematical analysis**.

Analysis of Algorithms in Rhymes & Pictures

When the math gets tough



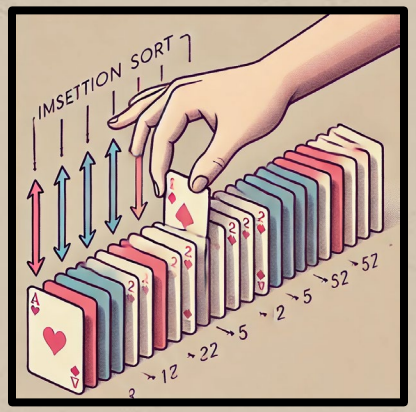
Analysis of Algorithms in Rhymes & Pictures

Insertion Sort: A Rhyming Tale

When sorting is what you need to do,
Insertion Sort will pull you through.
One by one, it takes each card,
And fits it in, though it seems hard.

The list is scanned from left to right,
And each new value finds its height.
Compared to those it left behind,
A better spot it soon will find.

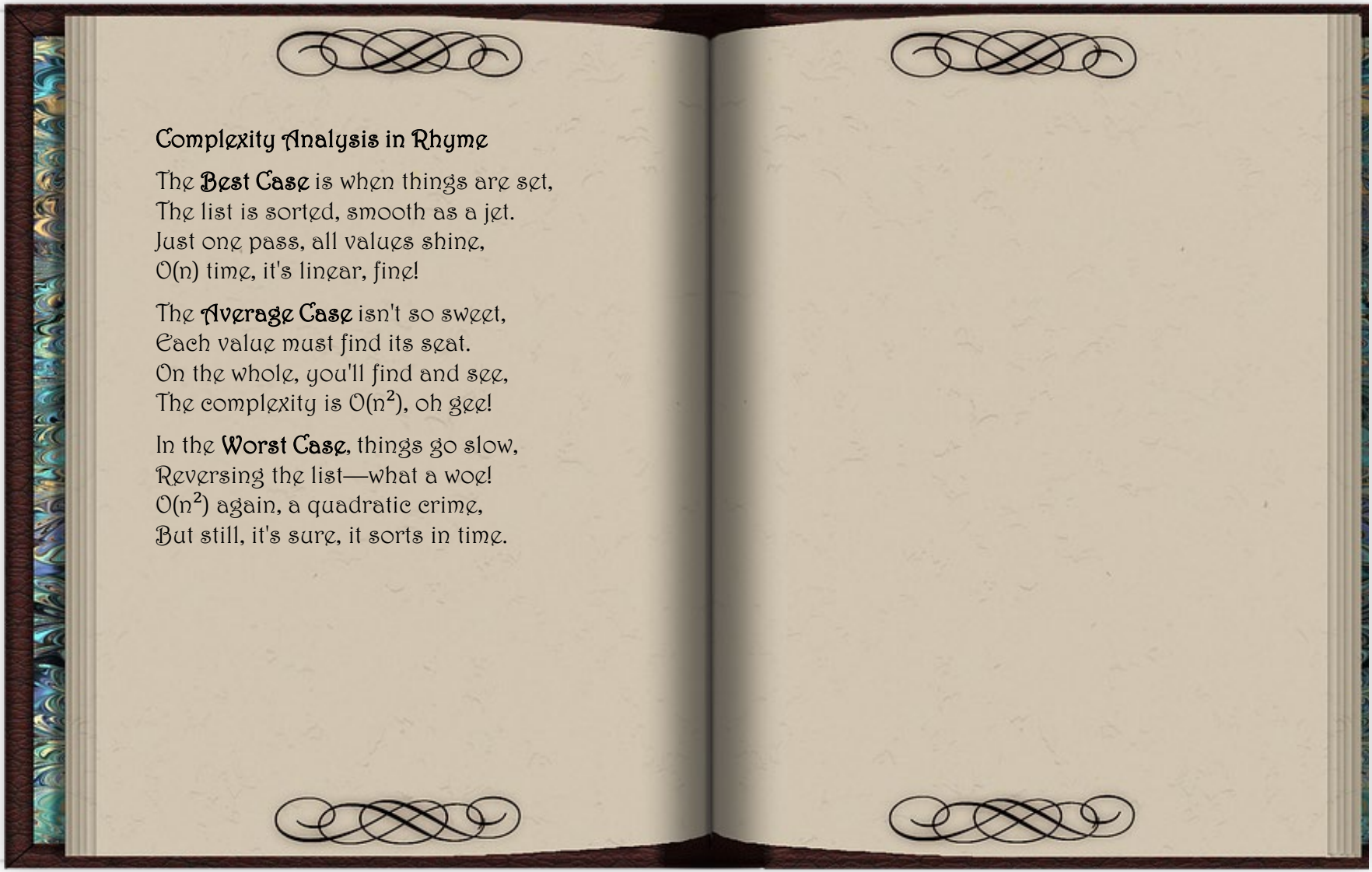
At first, it's slow, but don't despair,
With fewer moves, it's light as air.
For smaller lists, it works just fine,
Its worst is n-squared every time.



Here is an illustration of the insertion sort process. You can see how the hand is placing a new card in the correct position among already sorted cards, with arrows showing comparisons and movements. This image brings the sorting algorithm to life visually!

ChatGPT 4o mini

Analysis of Algorithms in Rhymes & Pictures



Complexity Analysis in Rhyme

The **Best Case** is when things are set,
The list is sorted, smooth as a jet.
Just one pass, all values shine,
O(n) time, it's linear, fine!

The **Average Case** isn't so sweet,
Each value must find its seat.
On the whole, you'll find and see,
The complexity is $O(n^2)$, oh gee!

In the **Worst Case**, things go slow,
Reversing the list—what a woe!
$O(n^2)$ again, a quadratic crime,
But still, it's sure, it sorts in time.

ChatGPT 4o mini

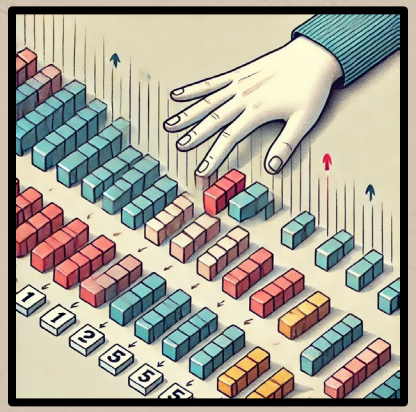
Analysis of Algorithms in Rhymes & Pictures

Selection Sort: A Rhyming Tale

When sorting's tough and you must choose,
Selection Sort will never lose.
It scans the list, it looks around,
For the smallest value to be found.

One by one, from start to end,
Each element must find a friend.
The smallest moves to the front row,
And then the rest continue to go.

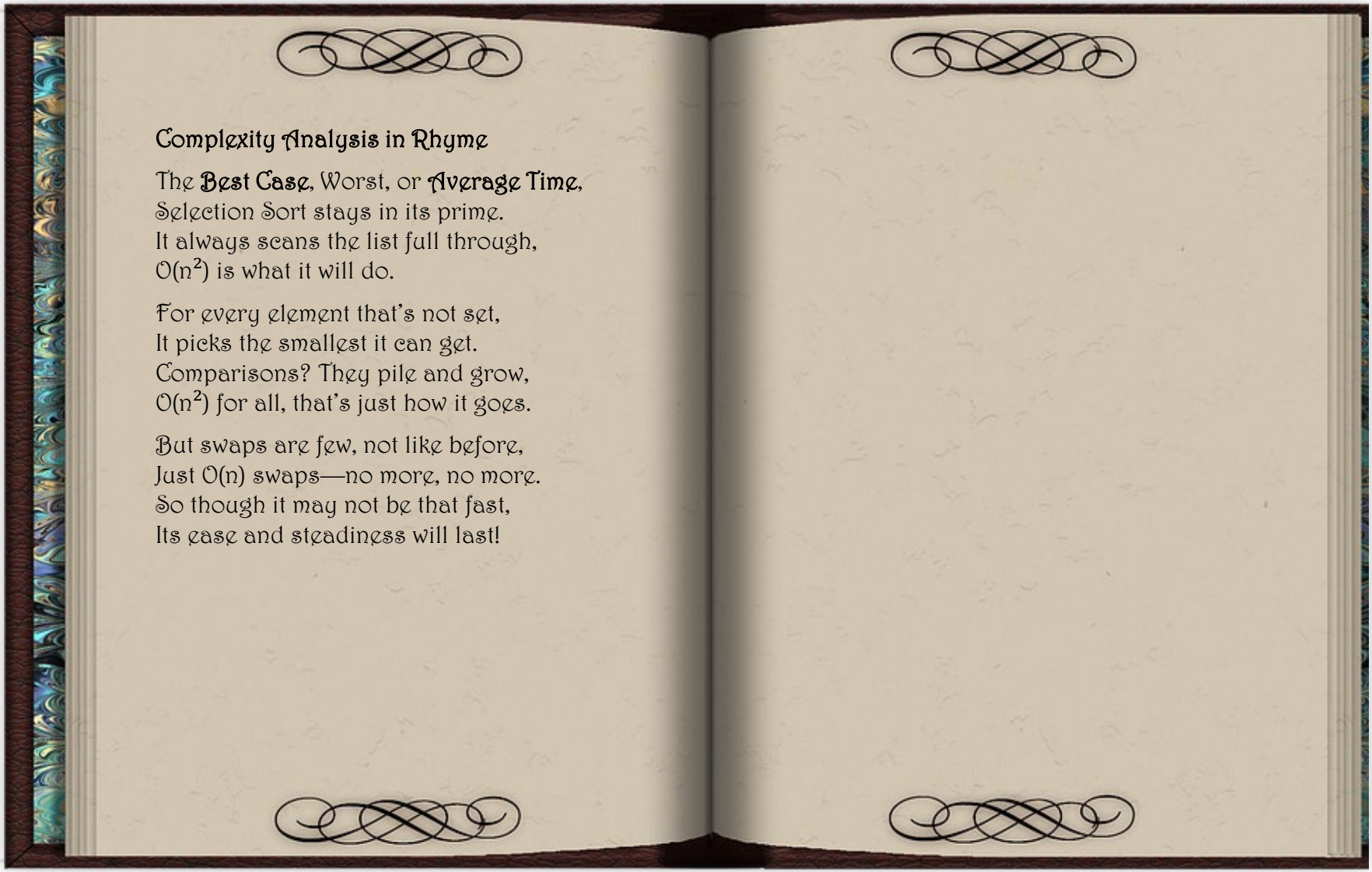
It swaps, it shifts, but moves are few,
For larger lists, it might be slow too.
Yet steady it works, no trick or ploy,
Its simple logic you'll enjoy.



Here's an illustration of selection sort in action. The hand is selecting the smallest block from the unsorted section on the right, preparing to swap it to the front, while the blocks on the left are already sorted. The process is shown with arrows to indicate the scanning and swapping.

ChatGPT 4o mini

Analysis of Algorithms in Rhymes & Pictures



Complexity Analysis in Rhyme

The **Best Case**, Worst, or **Average Time**,
Selection Sort stays in its prime.
It always scans the list full through,
$O(n^2)$ is what it will do.

For every element that's not set,
It picks the smallest it can get.
Comparisons? They pile and grow,
$O(n^2)$ for all, that's just how it goes.

But swaps are few, not like before,
Just $O(n)$ swaps—no more, no more.
So though it may not be that fast,
Its ease and steadiness will last!

ChatGPT 4o mini

Grading Policy

- Four Homework Problem Sets: 25%
  (worth 6.25% each)
  - Form groups of up to two for problem solving.
  - Each group will submit only one copy of their solutions through Brightspace.
  - Each group must report approximate % contribution of each member in solving each problem set.
- Midterm Exam (Monday, Mar 23, NCS 120, 6:30 – 7:50 pm): 25%
- Final Exam (Wednesday, May 13, NCS 120, 8:30 – 11:00 pm): 50%

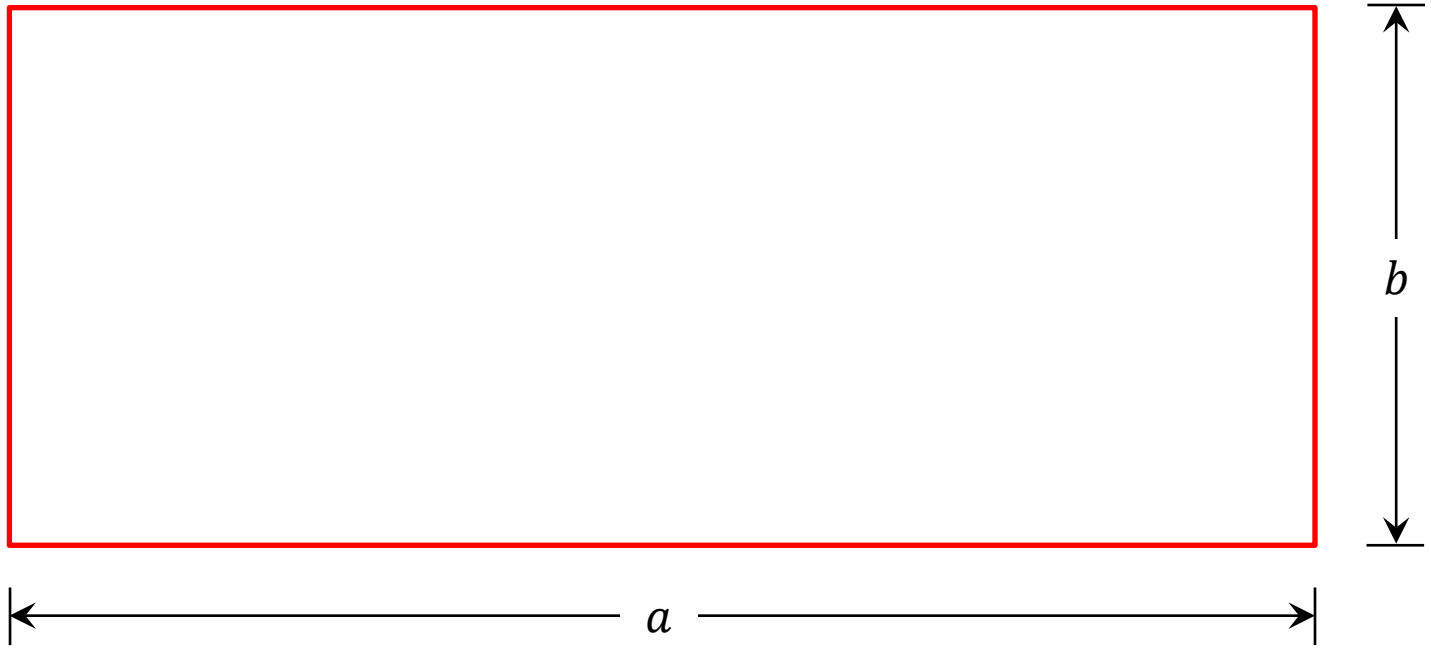
Textbooks

Recommended but not required.

- Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein.
  Introduction to Algorithms (3rd/4th Edition), MIT Press, 2009/2022.
- Sanjoy Dasgupta, Christos Papadimitriou, and Umesh Vazirani.
  Algorithms (1st Edition), McGraw-Hill, 2006.
- Jon Kleinberg and Éva Tardos.
  Algorithm Design (1st Edition), Addison Wesley, 2005.
- Steven Skiena.
  The Algorithm Design Manual (2nd /3rd Edition), Springer, 2008/2013.
- Pramod Ganapathi.
  Mathematical and Algorithmic Puzzles (Free Online Version), 2021.

A 2300+ Years Old Algorithm Still in Use Today

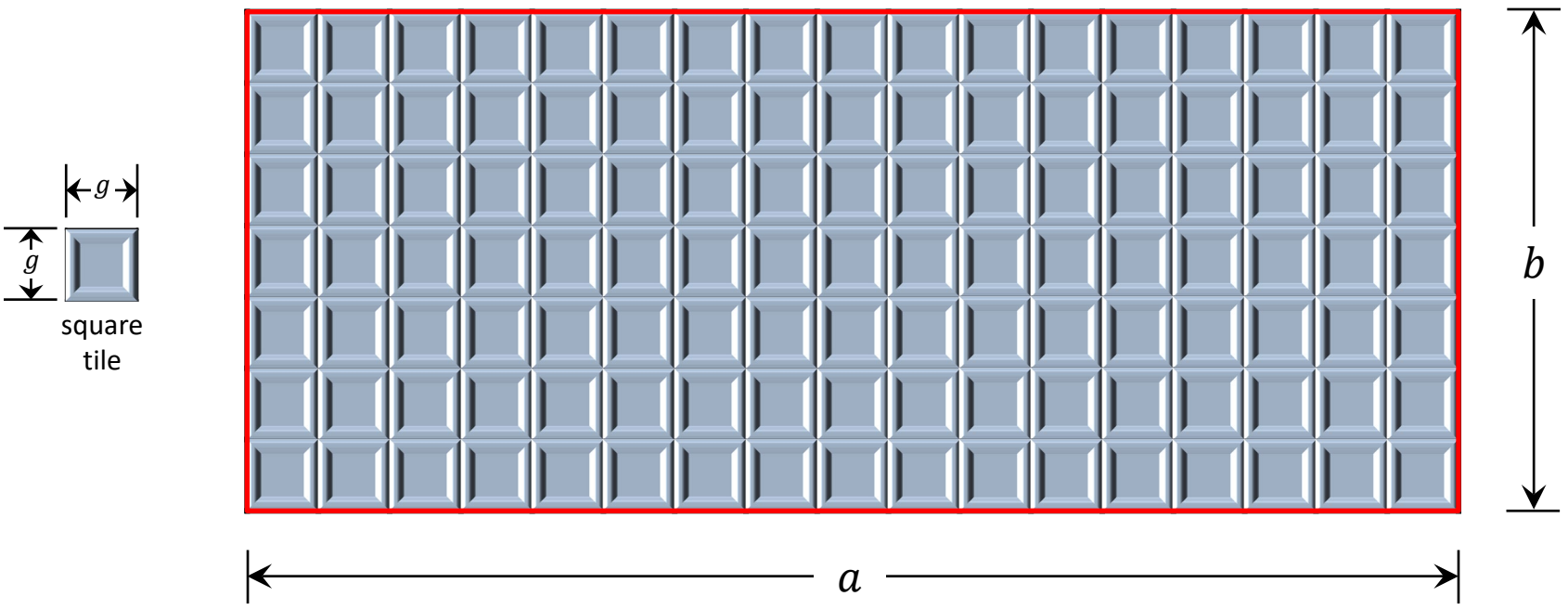
Input: A rectangular area of length a and width b,
where a and b are positive integers.



A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

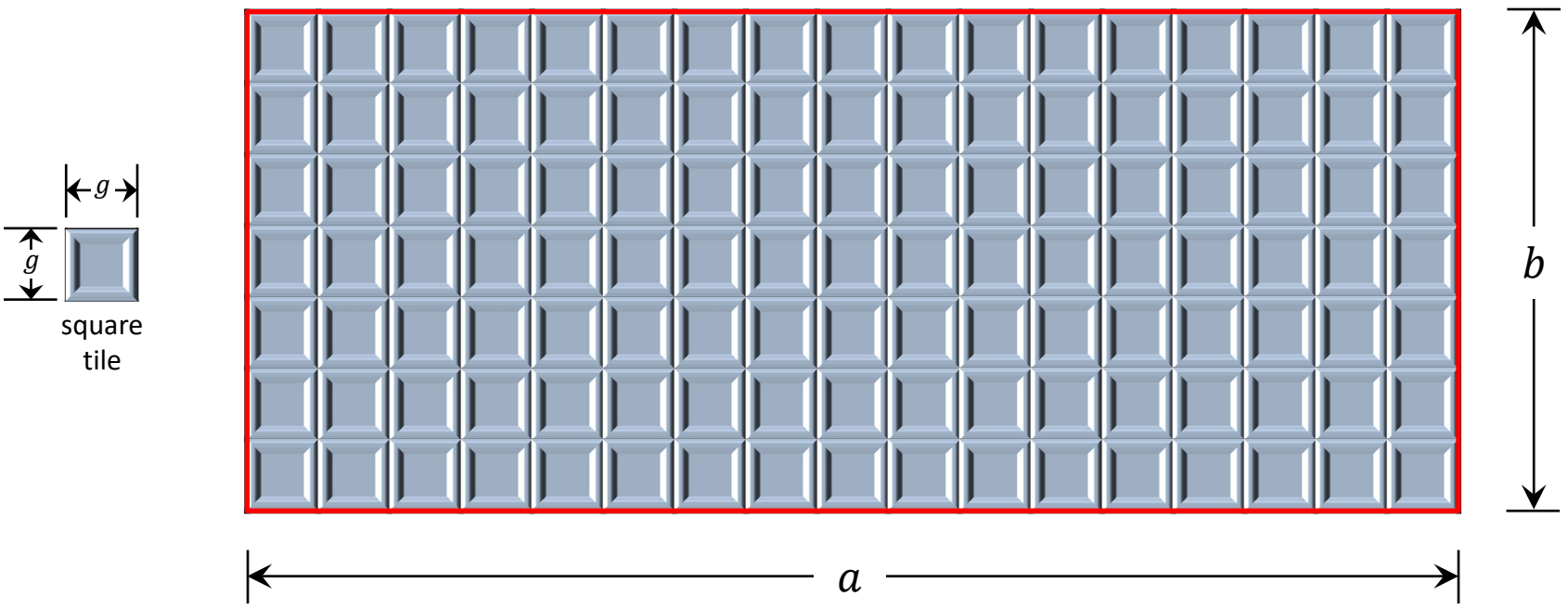


$$g = tile(a, b)$$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

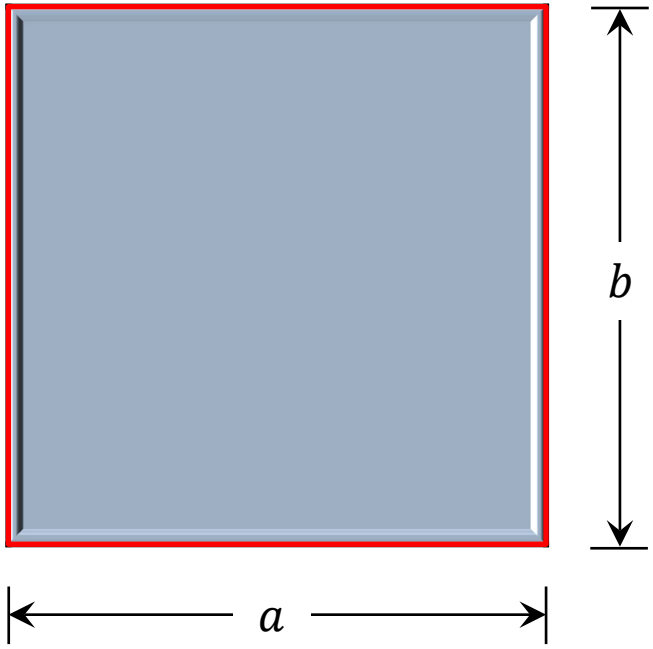


$if\ a = b\ then$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

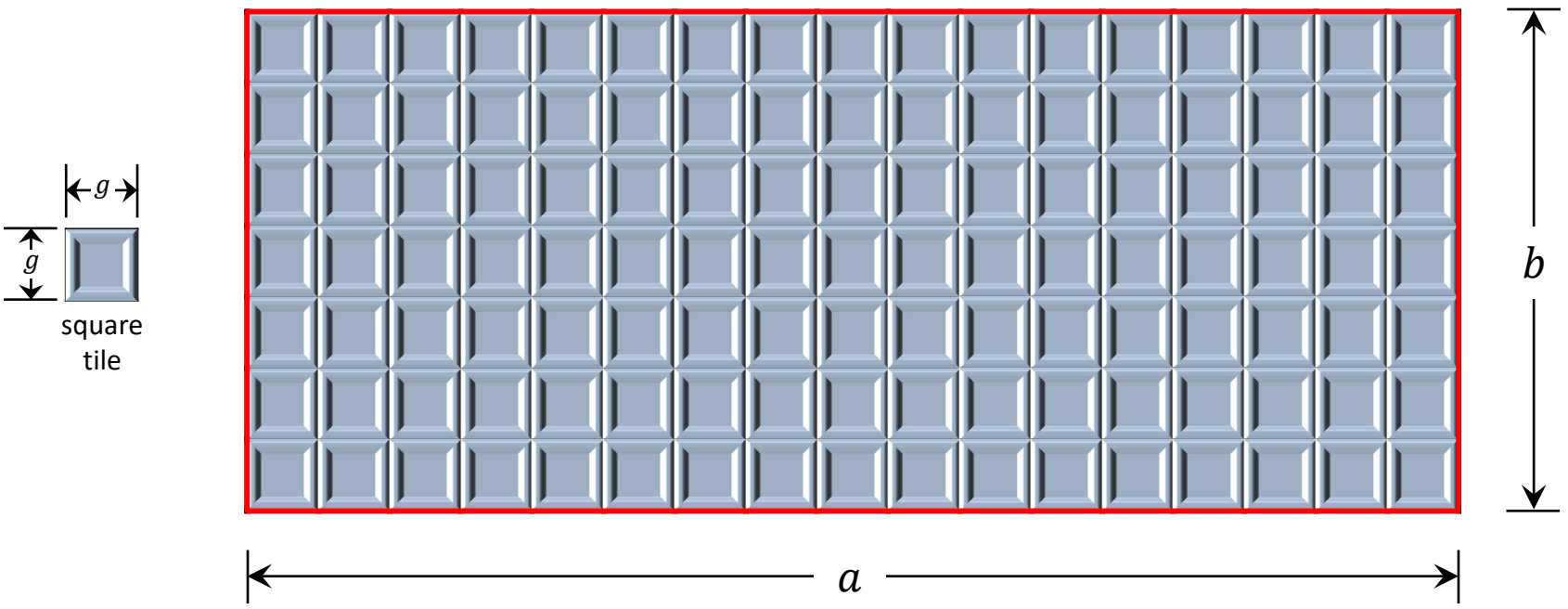


$$if\ a = b\ then$$
$$g = tile(a, b) = a = b$$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

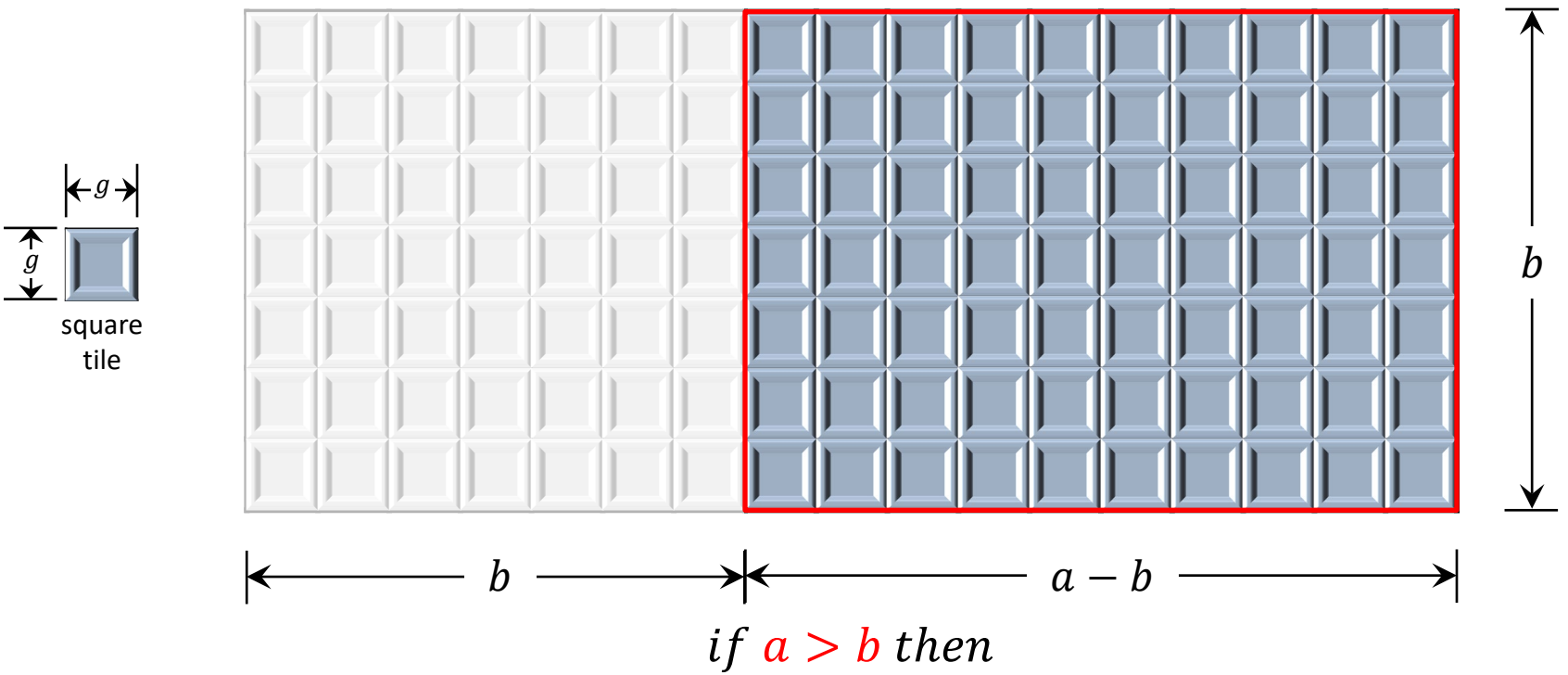


$$g = tile(a, b)$$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

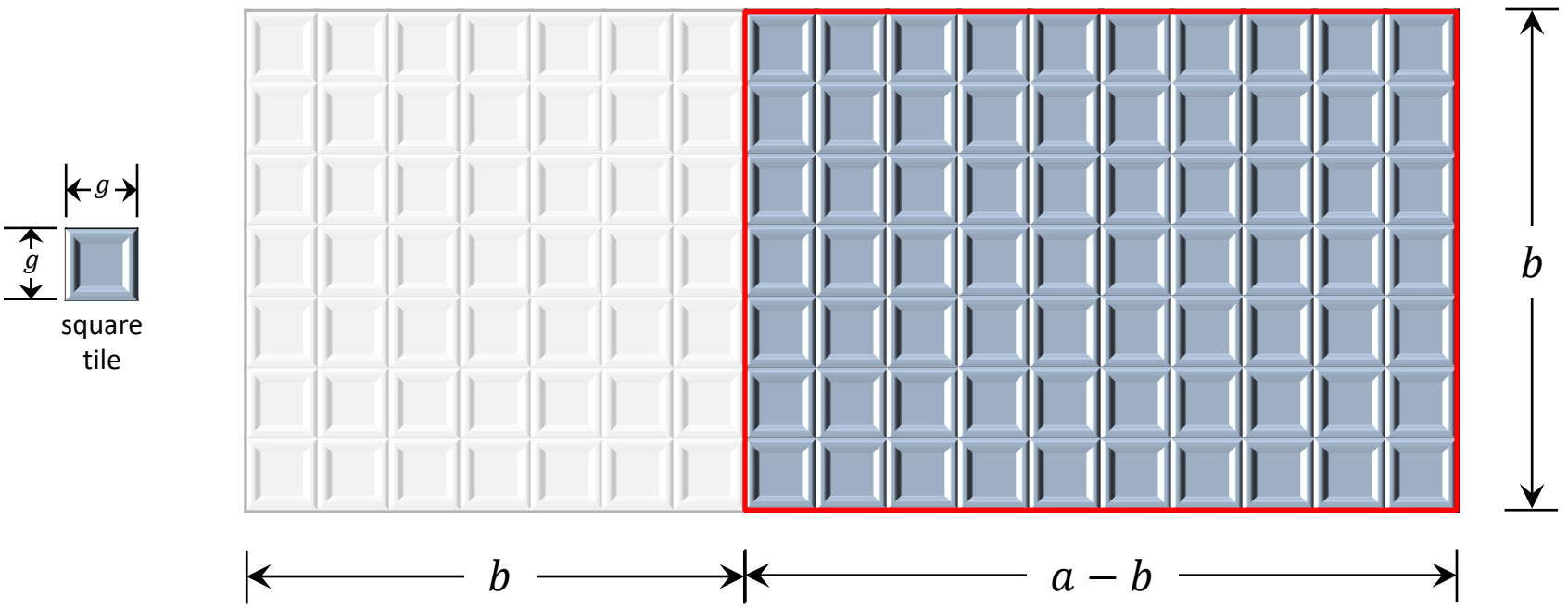
Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.



A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b, where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

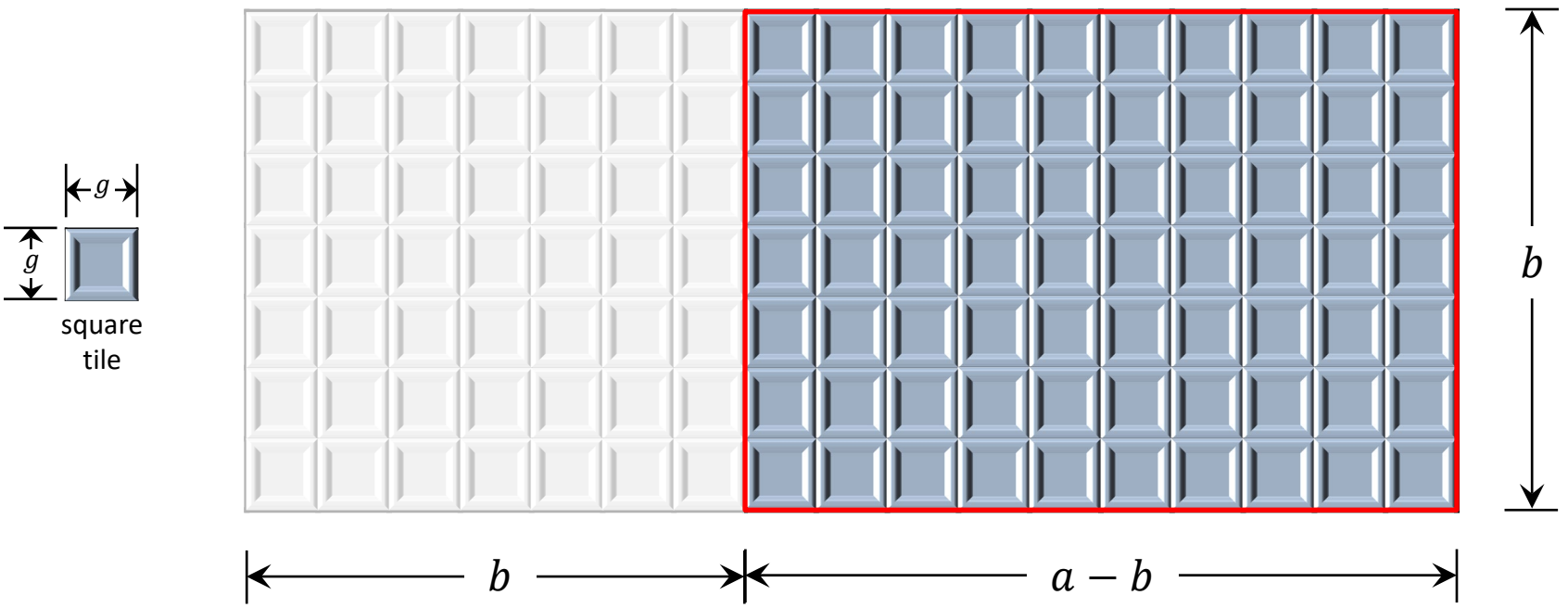


$$if\ a > b\ then$$
$$g = tile(a, b) = tile(a - b, b)$$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b, where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

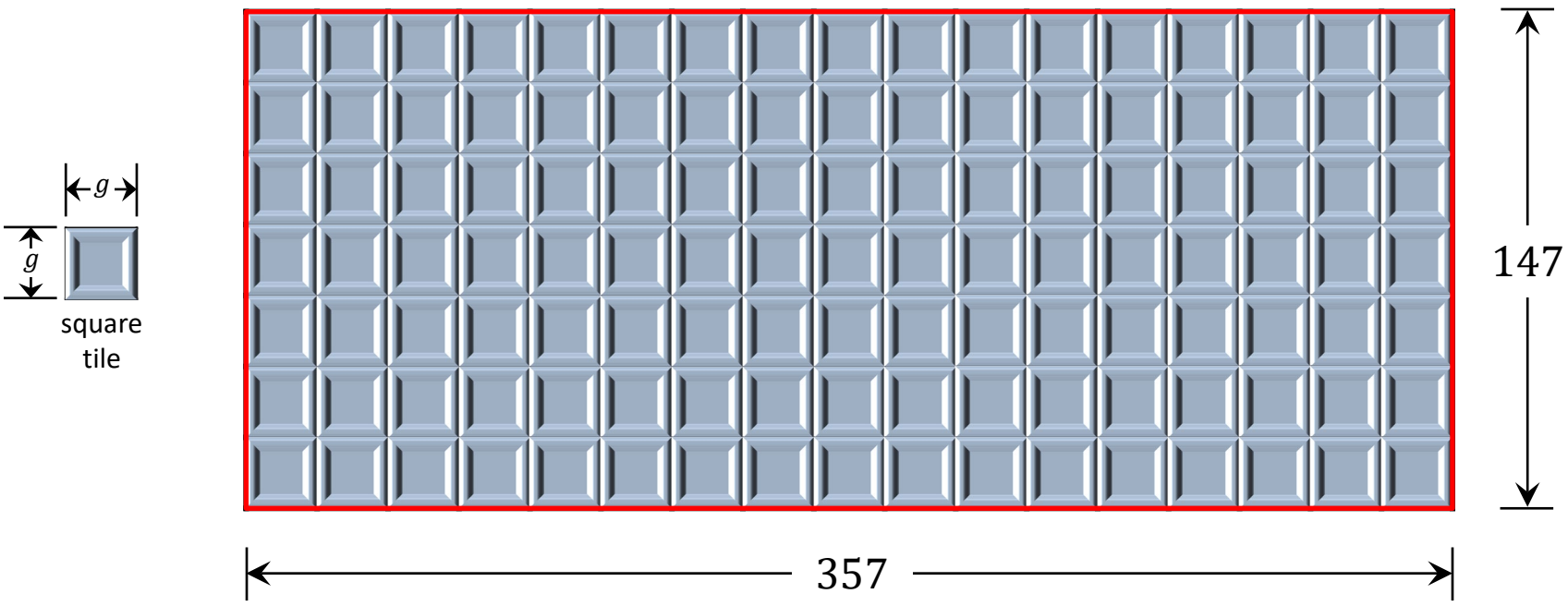


$$if\ a < b\ then$$
$$g = tile(a, b) = tile(a, b - a)$$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

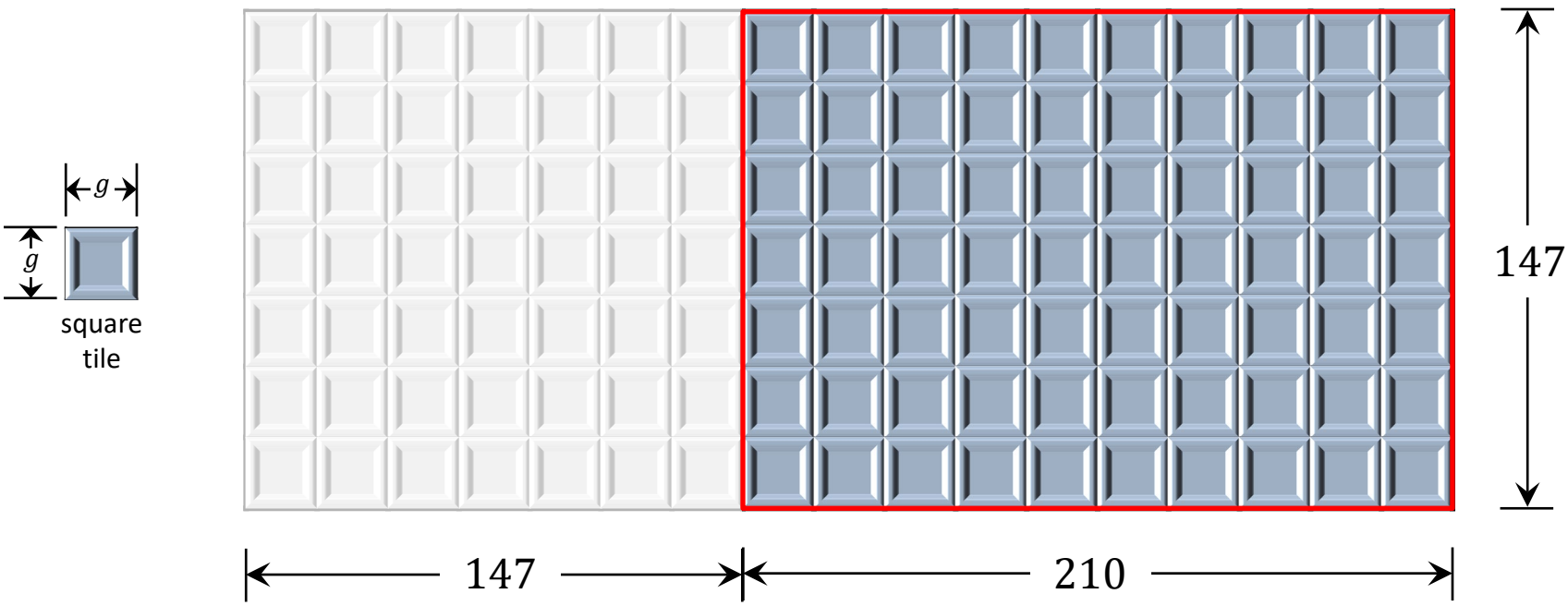


$g = tile(357,147)$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

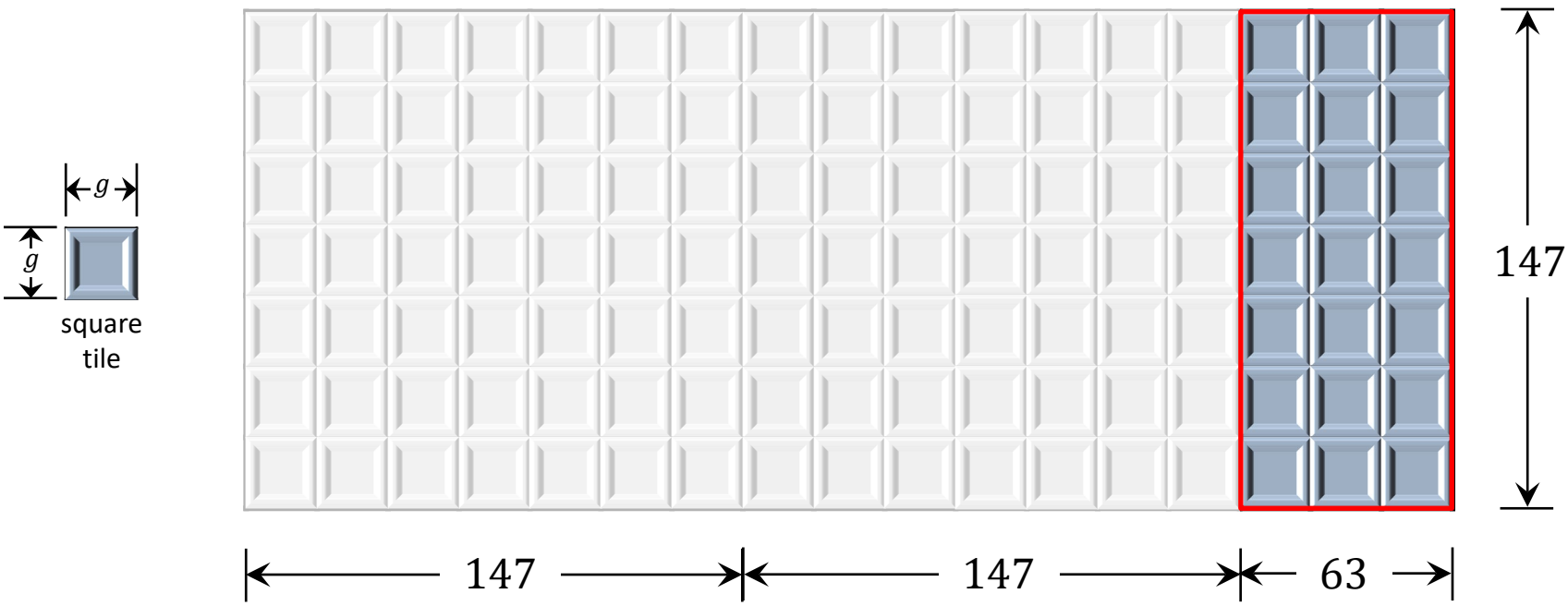


$g = tile(357,147) = tile(210,147)$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b, where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

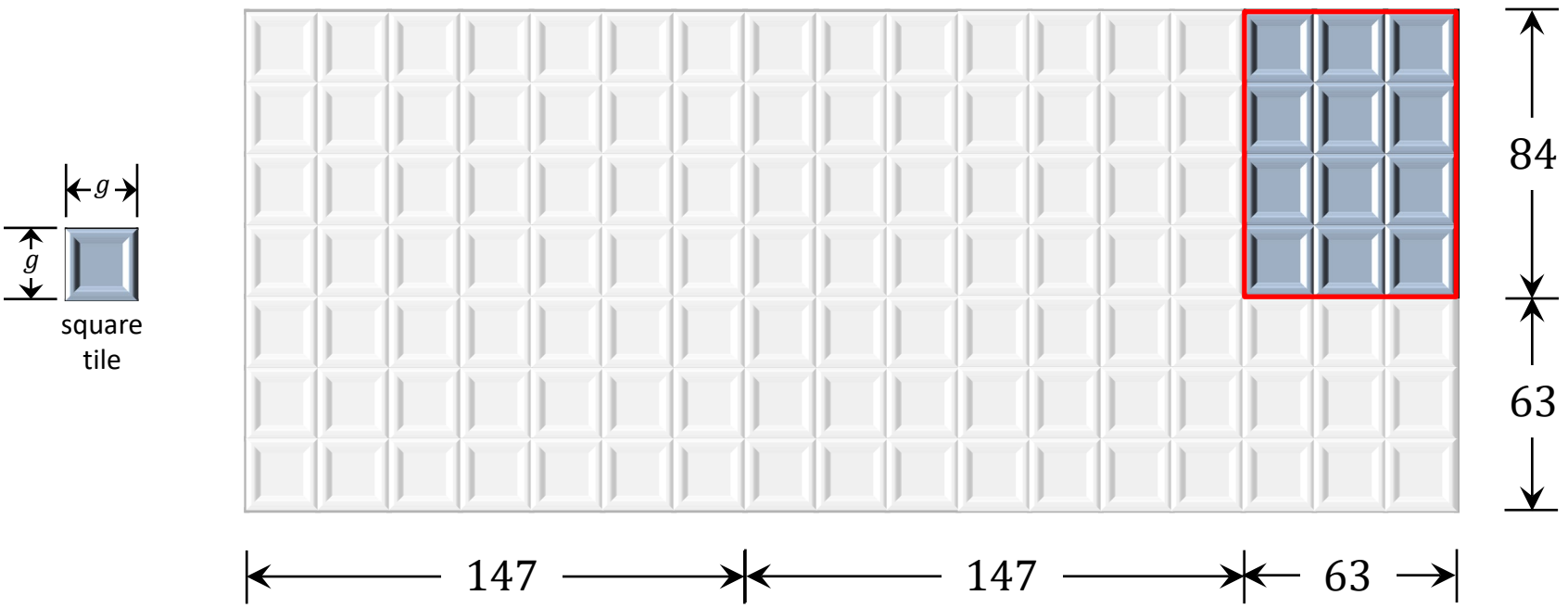


$g = tile(357,147) = tile(210,147) = tile(63,147)$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b, where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

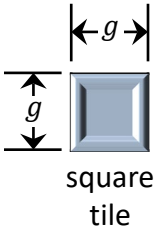


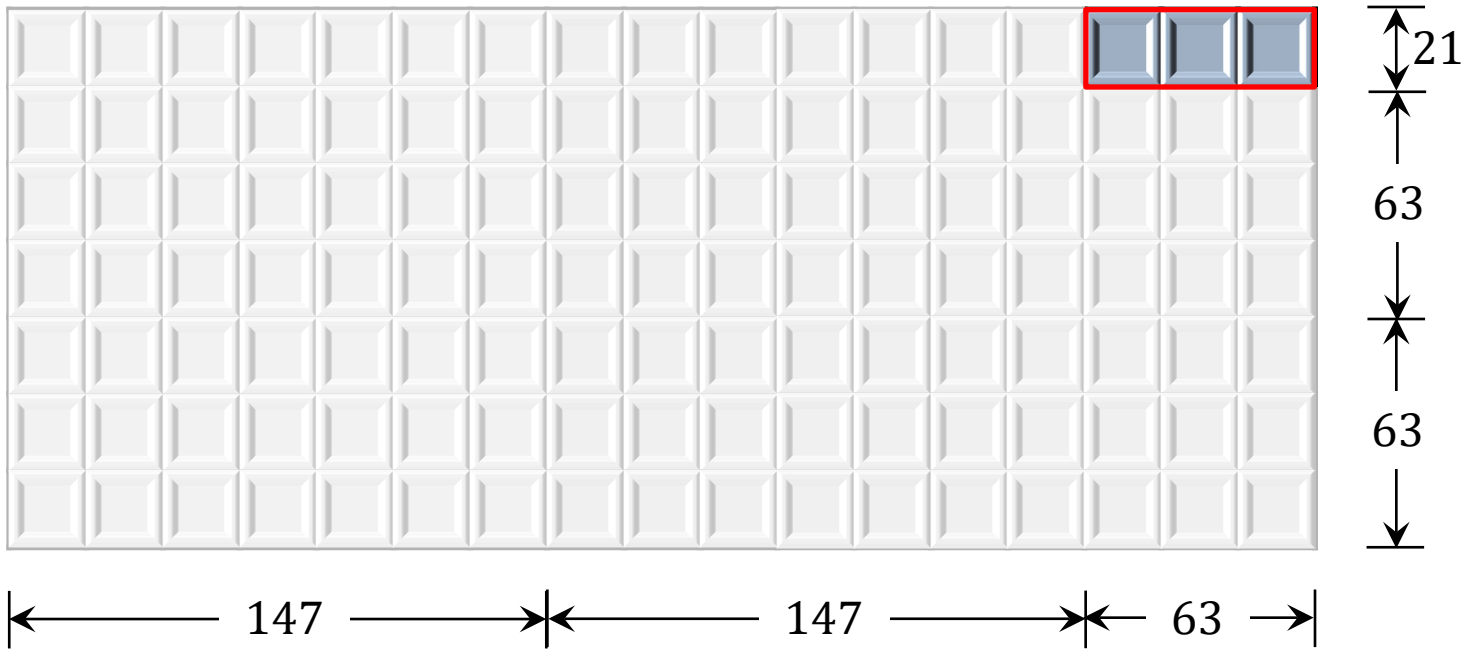
$$g = tile(357,147) = tile(210,147) = tile(63,147) = tile(63,84)$$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b, where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.



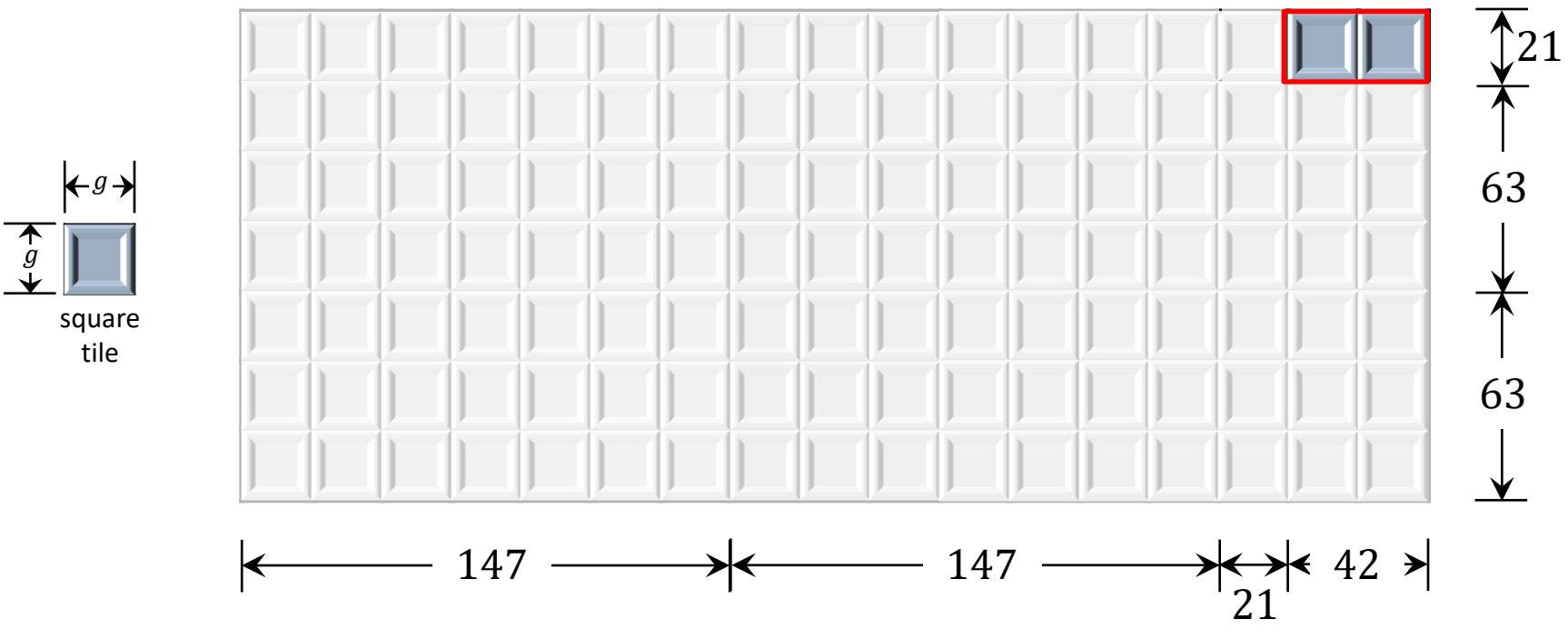


$g = tile(357,147) = tile(210,147) = tile(63,147) = tile(63,84)$
$= tile(63,21)$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b, where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.

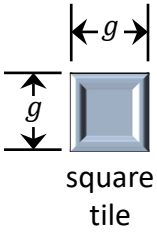


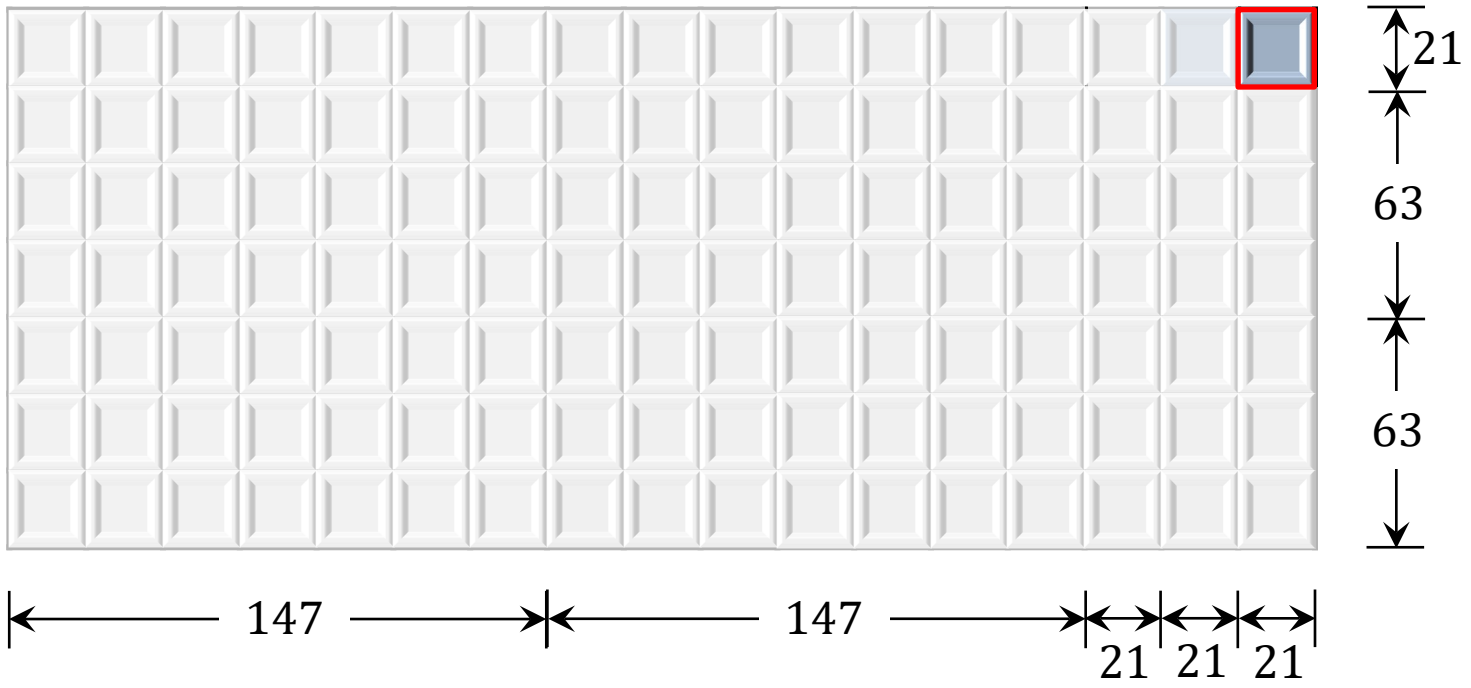
$g = tile(357,147) = tile(210,147) = tile(63,147) = tile(63,84)$
$= tile(63,21) = tile(42,21)$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.



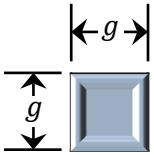


$g = tile(357,147) = tile(210,147) = tile(63,147) = tile(63,84)$
$= tile(63,21) = tile(42,21) = tile(21,21)$

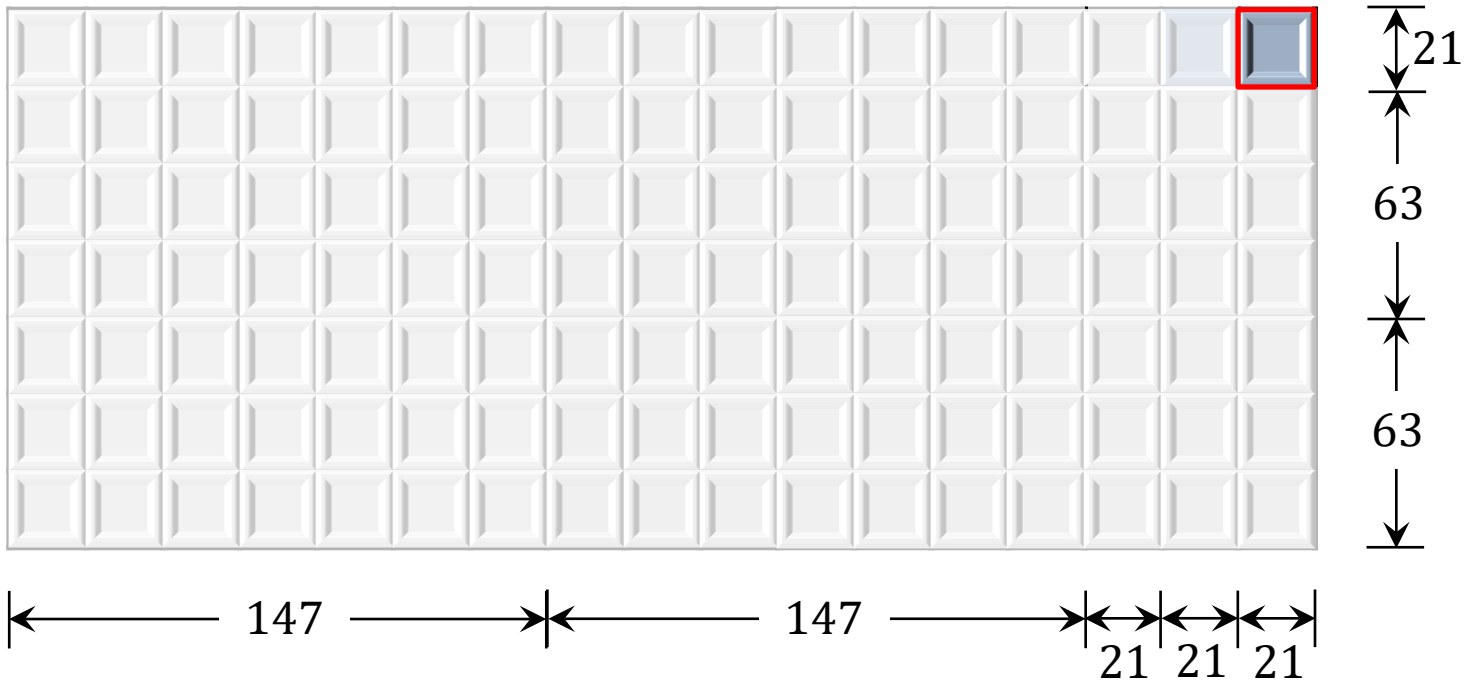
A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.



square tile

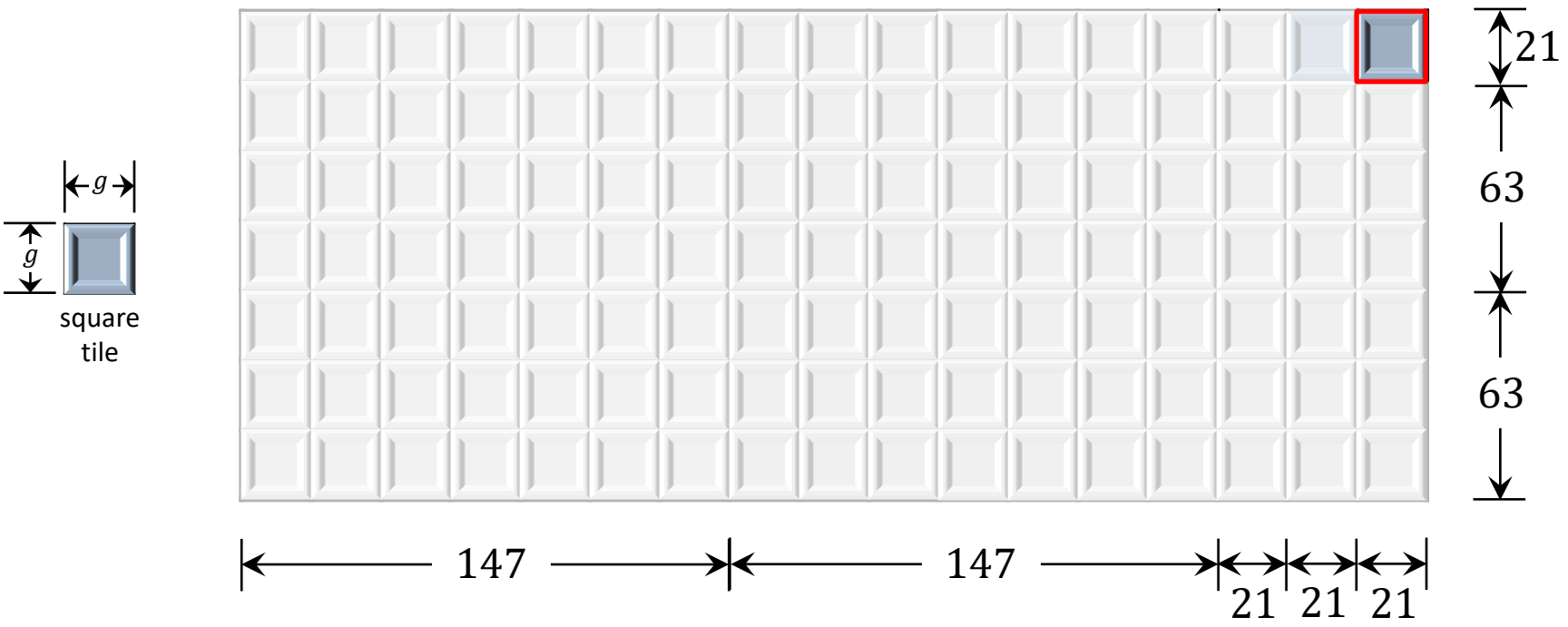


$g = tile(357,147) = tile(210,147) = tile(63,147) = tile(63,84)$
$= tile(63,21) = tile(42,21) = tile(21,21) = 21$

A 2300+ Years Old Algorithm Still in Use Today

Input: A rectangular area of length a and width b,
where a and b are positive integers.

Output: A positive integer g, where $g \times g$ is the largest square tile that covers the rectangle exactly without any overlap.



This is **Euclid's algorithm** for finding
the **greatest common divisor (gcd)** of two given natural numbers!

What is an Algorithm?

An algorithm is an ***explicit***, ***precise***, ***unambiguous***, ***mechanically-executable*** sequence of elementary instructions, intended to solve a ***well-specified computational problem***.

It accepts a value or set of values as ***input***, and produces a value or set of values as ***output.***

Example: *Selection Sort* solves the ***sorting problem*** specified as a relationship between the input and the output as follows.

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

What is an Algorithm?

Selection Sort solves the ***sorting problem*** specified as follows.

Input: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: A permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

The ***problem specification*** should include just enough detail that someone else could use the algorithm as a ***black box***, without knowing how and why the algorithm actually works.

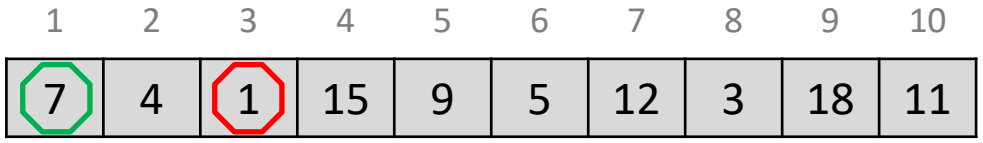
Input: A positive integer n and an array $A[\,1 : n\,]$ containing n numbers.

Output: Array $A[\,1 : n\,]$ with its original items rearranged in non-decreasing order of value.

Selection Sort

State 0

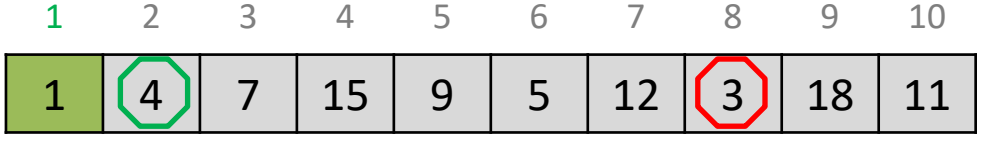
Input array **A**



Swap $A[1]$ with the smallest number in $A[1..10]$

State 1

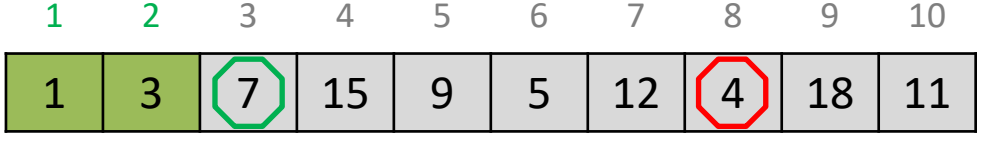
$A[1]$ is now sorted **A**



Swap $A[2]$ with the smallest number in $A[2..10]$

State 2

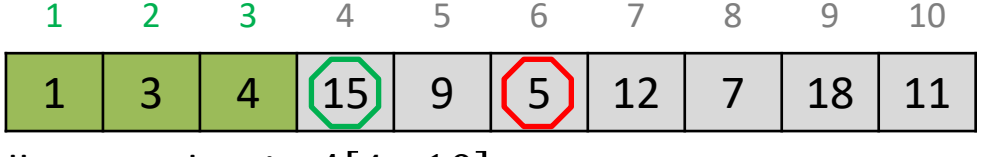
$A[1..2]$ is now sorted **A**



Swap $A[3]$ with the smallest number in $A[3..10]$

State 3

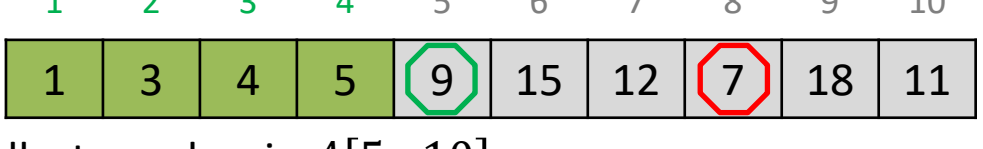
$A[1..3]$ is now sorted **A**



Swap $A[4]$ with the smallest number in $A[4..10]$

State 4

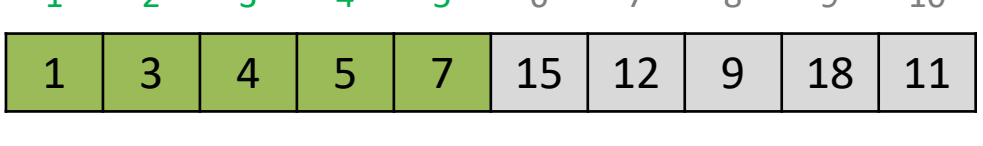
$A[1..4]$ is now sorted **A**



Swap $A[5]$ with the smallest number in $A[5..10]$

State 5

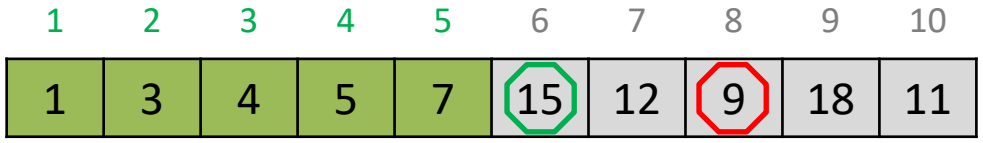
$A[1..5]$ is now sorted **A**



Selection Sort

State 5

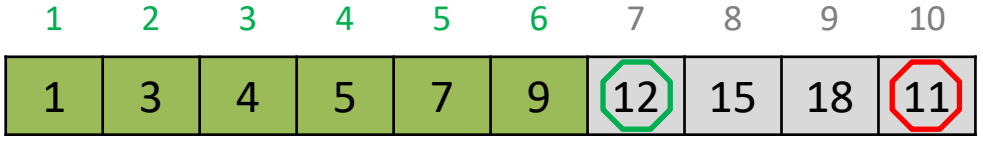
$A[1..5]$ is now sorted



Swap $A[6]$ with the smallest number in $A[6..10]$

State 6

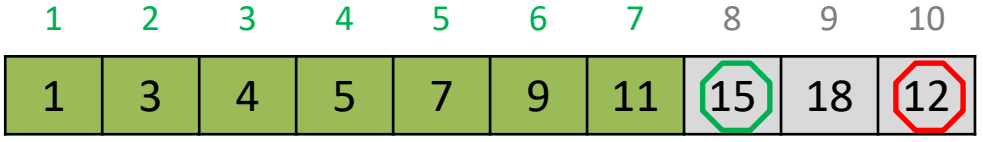
$A[1..6]$ is now sorted



Swap $A[7]$ with the smallest number in $A[7..10]$

State 7

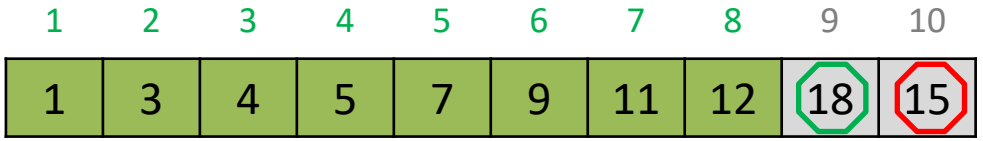
$A[1..7]$ is now sorted



Swap $A[8]$ with the smallest number in $A[8..10]$

State 8

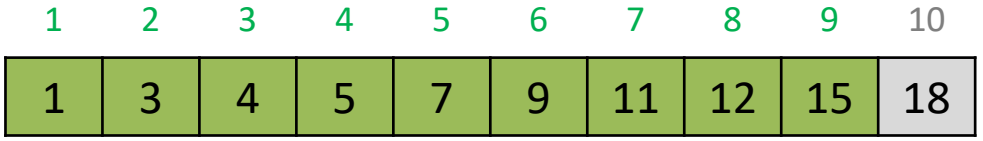
$A[1..8]$ is now sorted



Swap $A[9]$ with the smallest number in $A[9..10]$

State 9

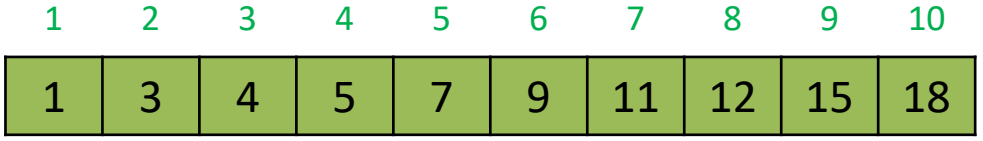
$A[1..9]$ is now sorted



Do nothing!

State 10

$A[1..10]$ is now sorted



Selection Sort

Input: A positive integer n and an array $A[\,1 : n\,]$ containing n numbers.

Output: Array $A[\,1 : n\,]$ with its original items rearranged in non-decreasing order of value.

SELECTION-SORT (*n*, *A*)

```
1.  for j = 1 to n
2.      // find the index of an entry with the smallest value in A[j..A.length]
3.      min = j
4.      for i = j + 1 to n
5.          if A[i] < A[min]
6.              min = i
7.      // swap A[j] and A[min]
8.      A[j] ↔ A[min]
```

Insertion Sort

Input: A positive integer n and an array $A[\,1 : n\,]$ containing n numbers.

Output: Array $A[\,1 : n\,]$ with its original items rearranged in non-decreasing order of value.

INSERTION-SORT (*n*, *A*)

1. ***for*** $j = 2$ ***to*** n
2. $key = A[j]$
3. // insert $A[j]$ into the sorted sequence $A[1..j-1]$
4. $i = j - 1$
5. ***while*** $i > 0$ *and* $A[i] > key$
6. $A[i+1] = A[i]$
7. $i = i - 1$
8. $A[i+1] = key$

Desirable Properties of an Algorithm

√ Correctness

- Designing an incorrect algorithm is straightforward

√ Efficiency

- Efficiency is easily achievable if we give up on correctness

Surprisingly, sometimes algorithms producing incorrect output can also be useful!

- If you can control the error rate
- Tradeoff between correctness and efficiency:

  Randomized algorithms

  (Monte Carlo: always efficient but sometimes incorrect/fails,

  Las Vegas: always correct but sometimes inefficient)

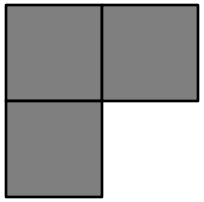
  Approximation algorithms

  (usually efficient but incorrect!)

Algorithmic Puzzles

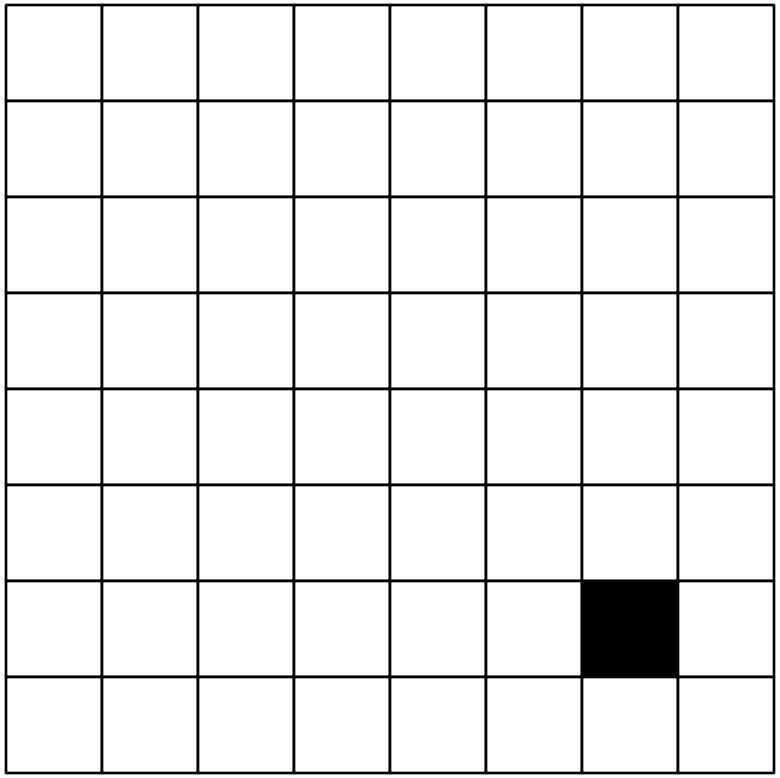
Tromino Cover

A *right tromino* is an L-shaped tile formed by three adjacent squares.



Puzzle: You are given a $2^n \times 2^n$ board with one missing square.

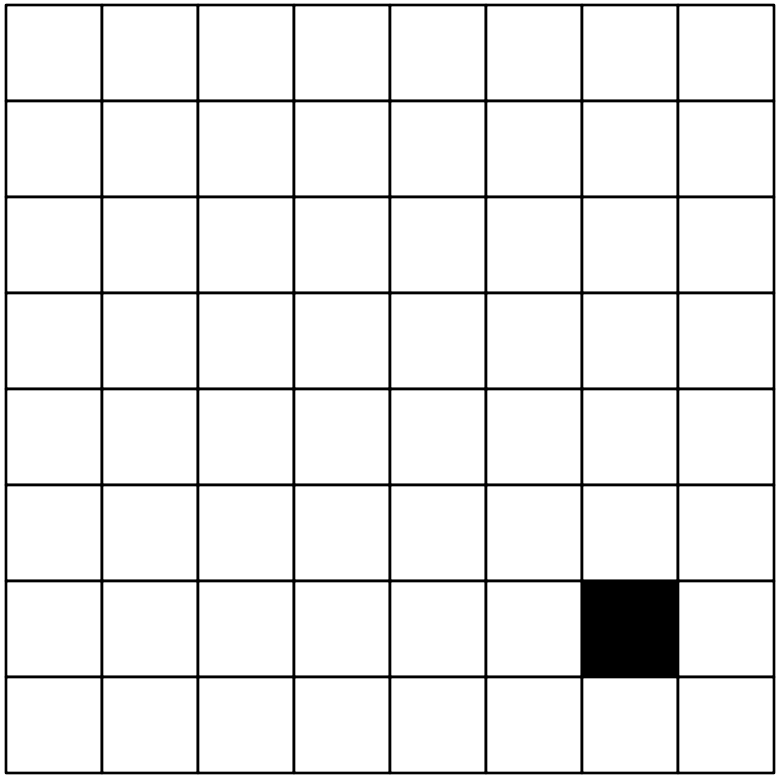
- you must cover all squares except the missing one exactly using right trominoes
- the trominoes must not overlap



$2^3 \times 2^3$ board

Tromino Cover

Steps

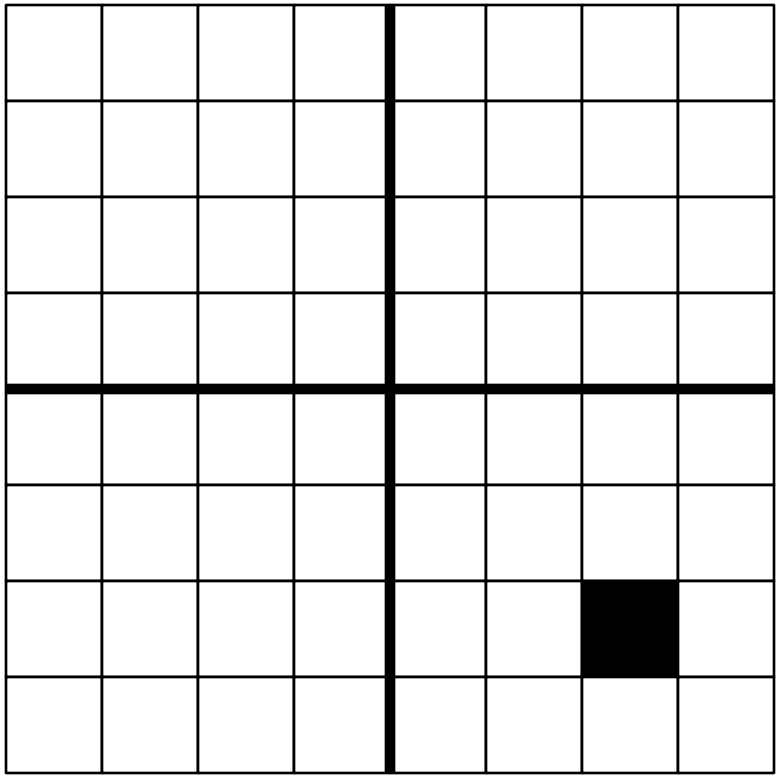


$2^3 \times 2^3$ board

Tromino Cover

Steps

– Divide the $2^n \times 2^n$ board into 4 disjoint $2^{n-1} \times 2^{n-1}$ subboards.

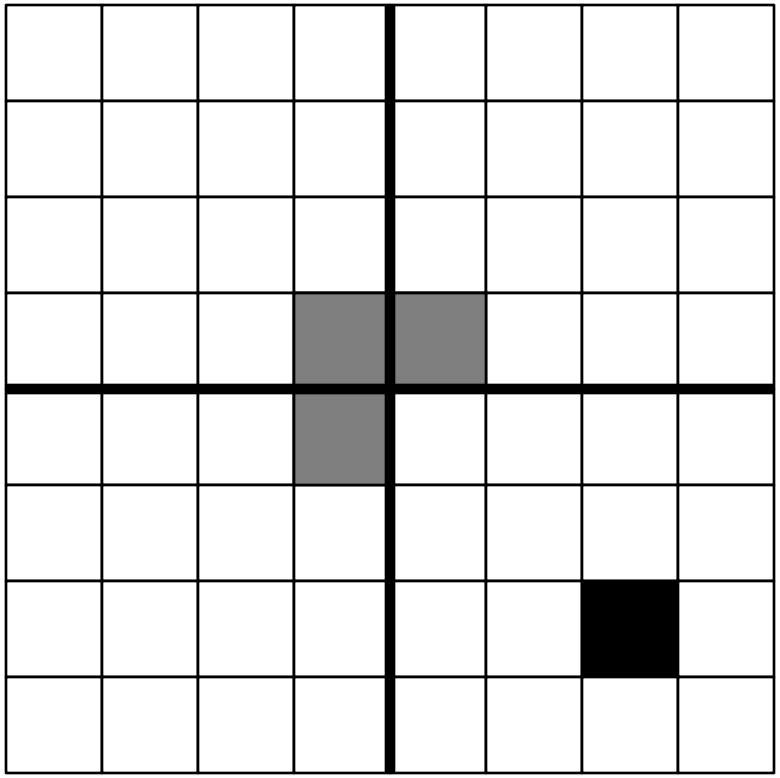


$2^3 \times 2^3$ board

Tromino Cover

Steps

- Divide the $2^n \times 2^n$ board into 4 disjoint $2^{n-1} \times 2^{n-1}$ subboards.
- Place a tromino at the center so that it fully covers one square from each of the three (3) subboards with no missing square, and misses the fourth subboard completely.



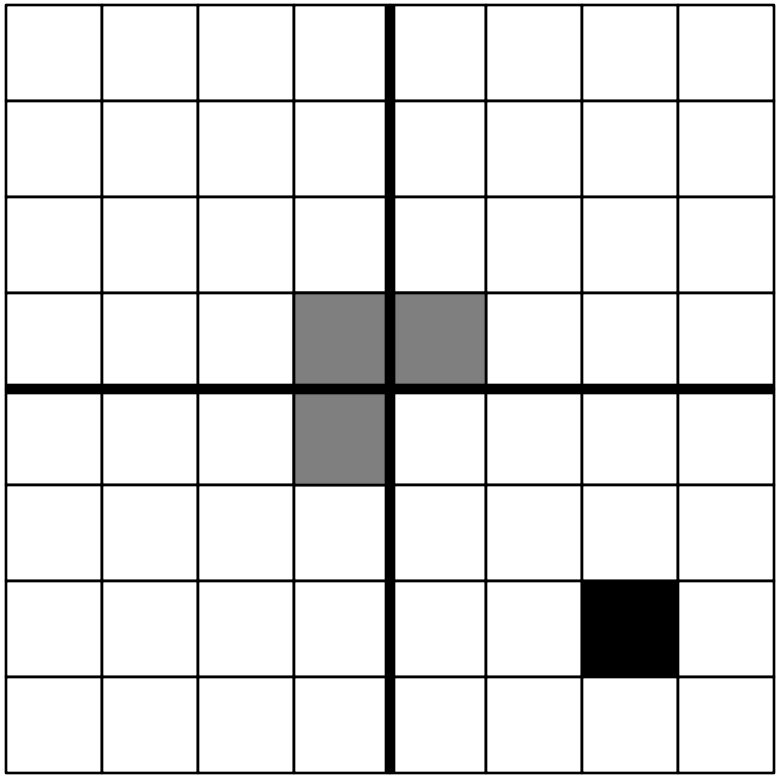
$2^3 \times 2^3$ board

Tromino Cover

Steps

- Divide the $2^n \times 2^n$ board into 4 disjoint $2^{n-1} \times 2^{n-1}$ subboards.
- Place a tromino at the center so that it fully covers one square from each of the three (3) subboards with no missing square, and misses the fourth subboard completely.

  This reduces the original problem into 4 smaller instances of the same problem!



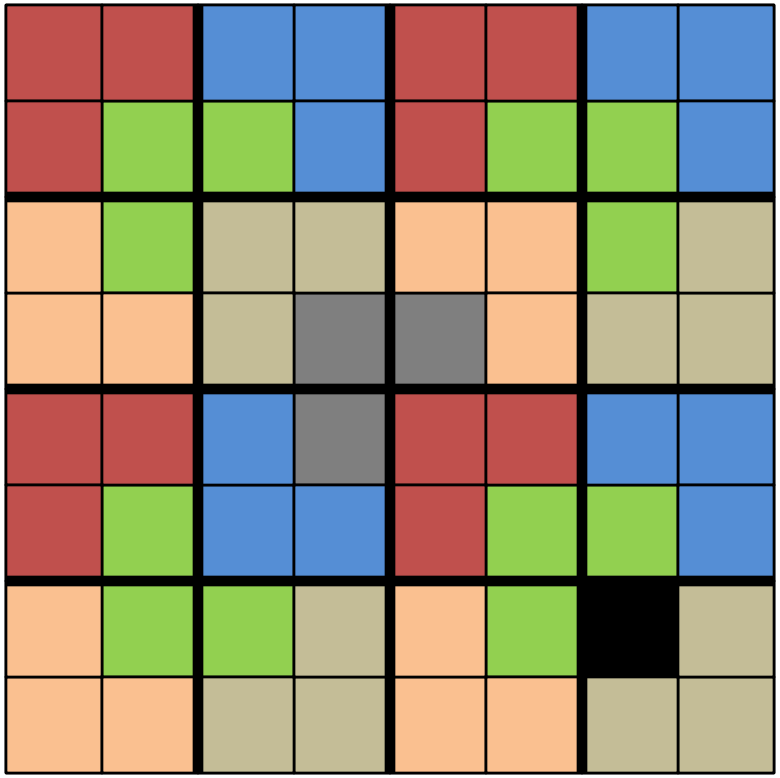
$2^3 \times 2^3$ board

Tromino Cover

Steps

- Divide the $2^n \times 2^n$ board into 4 disjoint $2^{n-1} \times 2^{n-1}$ subboards.
- Place a tromino at the center so that it fully covers one square from each of the three (3) subboards with no missing square, and misses the fourth subboard completely.

  This reduces the original problem into 4 smaller instances of the same problem!
- Solve each smaller subproblem recursively using the same technique.



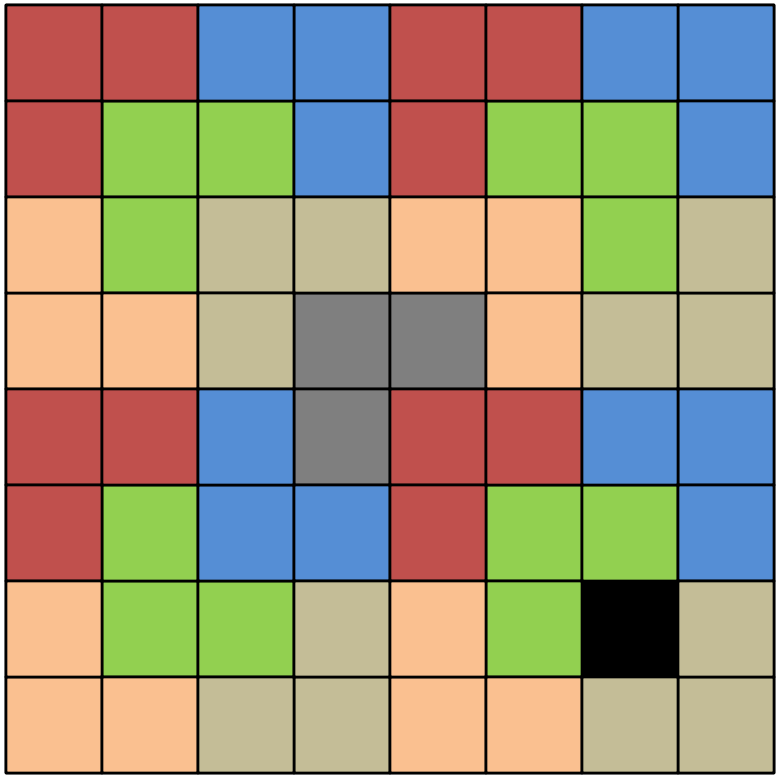
$2^3 \times 2^3$ board

Tromino Cover

Steps

- Divide the $2^n \times 2^n$ board into 4 disjoint $2^{n-1} \times 2^{n-1}$ subboards.
- Place a tromino at the center so that it fully covers one square from each of the three (3) subboards with no missing square, and misses the fourth subboard completely.

  This reduces the original problem into 4 smaller instances of the same problem!



$2^3 \times 2^3$ board

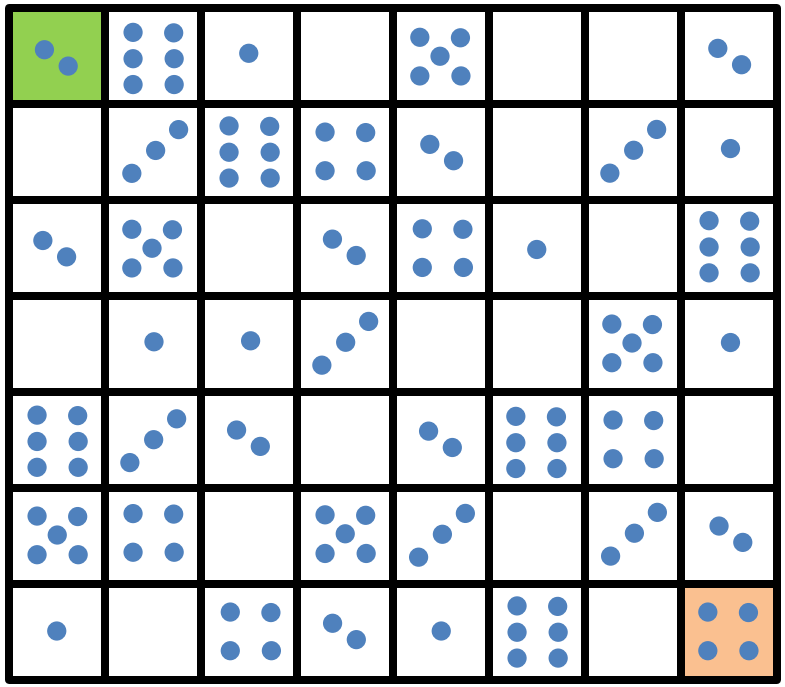
- Solve each smaller subproblem recursively using the same technique.
- This algorithm design technique is called *recursive divide & conquer.*

Collecting Coins

Puzzle: A robot moves from the top-left corner to the bottom-right corner of an $m \times n$ grid. At each step it either moves one cell to the right or one cell down from its current location.

Each cell of the grid contains zero or more coins. The robot collects the coins from every cell it visits.

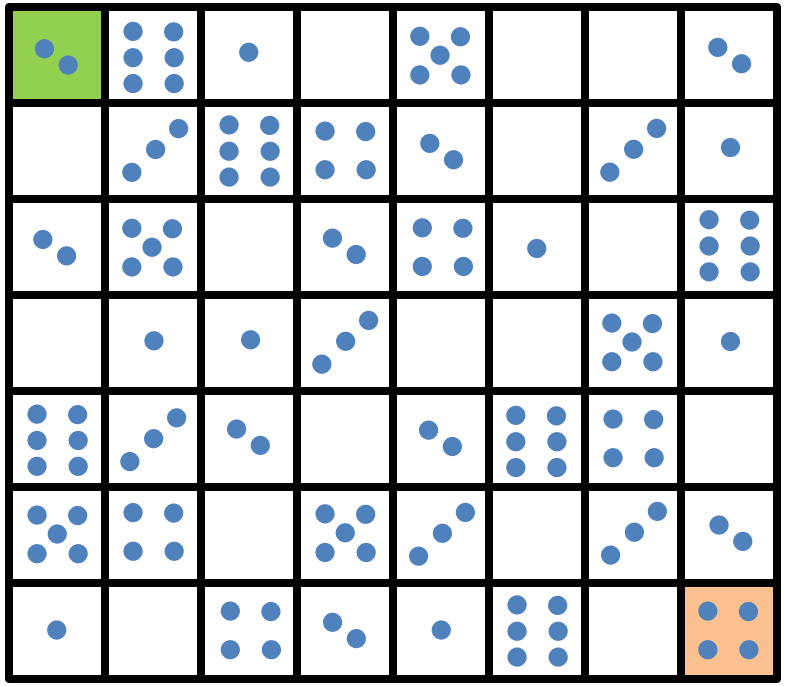
What path the robot should take to collect the maximum number of coins?

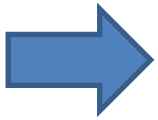


8×7 grid

Collecting Coins

Let us first count the number of coins in each cell.





C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

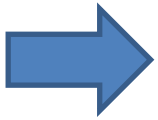
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

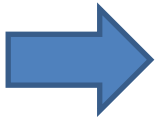
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2 →	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

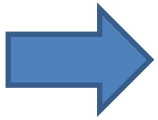
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

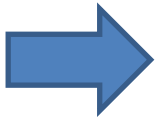
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8 →	1	0	5	0	0	2
2	2	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

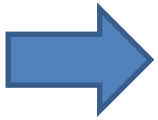
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	0	5	0	0	2
2	2	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

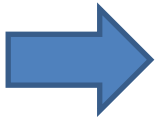
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	0	5	0	0	2
2	2	11	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

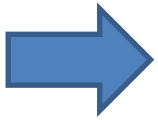
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9 → 0		5	0	0	2
2	2	11	6	4	2	0	3	1
3	4	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

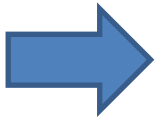
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	5	0	0	2
2	2	11	6	4	2	0	3	1
3	4	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

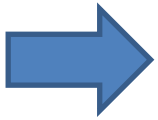
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	5	0	0	2
2	2	11	17	4	2	0	3	1
3	4	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

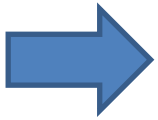
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	5	0	0	2
2	2	11	17	4	2	0	3	1
3	4	16	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

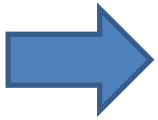
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	5	0	0	2
2	2	11	17	4	2	0	3	1
3	4	16	0	2	4	1	0	6
4	4	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

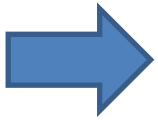
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

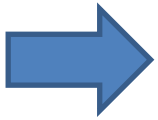
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

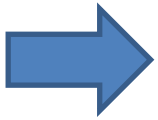
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell $[1,1]$ to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

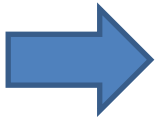
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

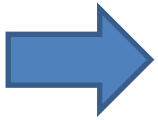
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

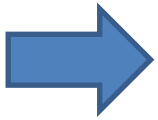
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

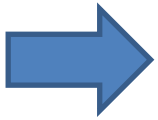
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

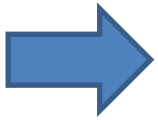
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

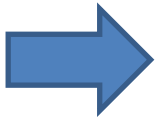
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

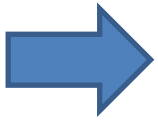
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

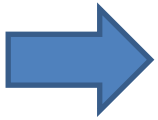
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

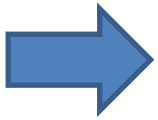
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4



C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

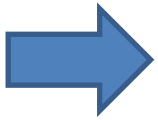
Collecting Coins

Let $c_{i,j}$ = number of coins in cell $[i,j]$

Let $C[i,j]$ = max number of coins the robot can collect if it goes from cell [1,1] to cell $[i,j]$

$$C[i,j] = \begin{cases} 0, & if\ i < 0\ or\ j < 0 \\ \max\{C[i,j-1], C[i-1,j]\} + c_{i,j}, & otherwise \end{cases}$$

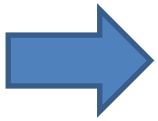
C	1	2	3	4	5	6	7	8
1	2	6	1	0	5	0	0	2
2	0	3	6	4	2	0	3	1
3	2	5	0	2	4	1	0	6
4	0	1	1	3	0	0	5	1
5	6	3	2	0	2	6	4	0
6	5	4	0	5	3	0	3	2
7	1	0	4	2	1	6	0	4

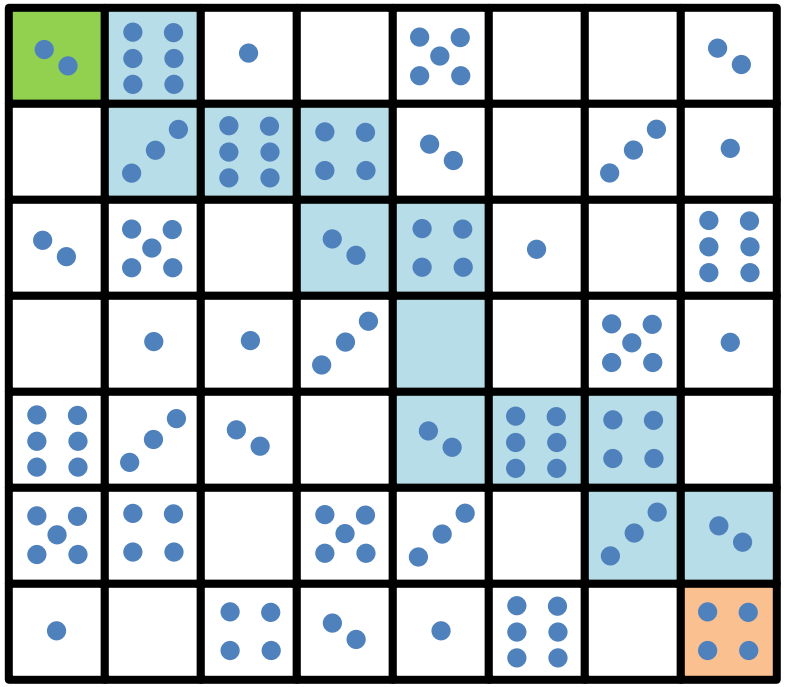


C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48

Collecting Coins

C	1	2	3	4	5	6	7	8
1	2	8	9	9	14	14	14	16
2	2	11	17	21	23	23	26	27
3	4	16	17	23	27	28	28	34
4	4	17	18	26	27	28	33	35
5	10	20	22	26	29	35	39	39
6	15	24	24	31	34	35	42	44
7	16	24	28	33	35	41	42	48





This algorithm design technique is called *dynamic programming*.

We computed solutions to larger and larger instances of the given problem using saved solutions to smaller overlapping instances of the same problem until we found a solution to the given instance.

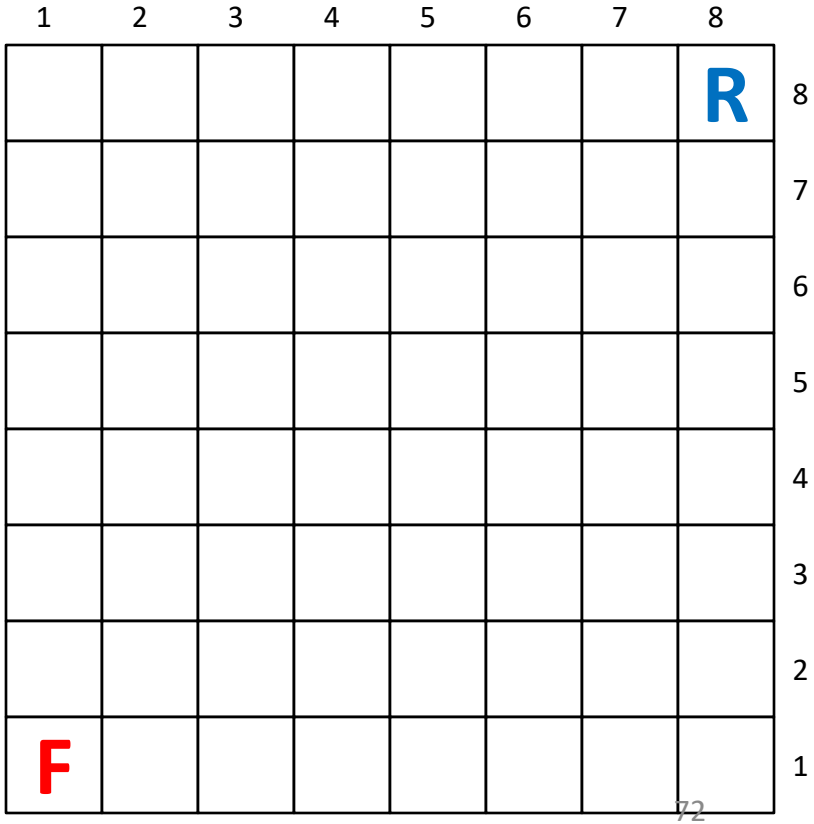
Rooster Chase

Puzzle: A farmer (**F**) is trying to catch a rooster (**R**) on a $n \times n$ grid.

- **F**'s initial position: bottom-left corner of the grid
- **R**'s initial position: top-right corner of the grid
- **F** and **R** move alternately until **R** is captured
- each move is to a neighboring cell horizontally or vertically
- **R** is captured when **F** moves to a cell occupied by **R**

What algorithm should **F** use to catch **R** when

- **F** moves first?
- **R** moves first?



8×8 board

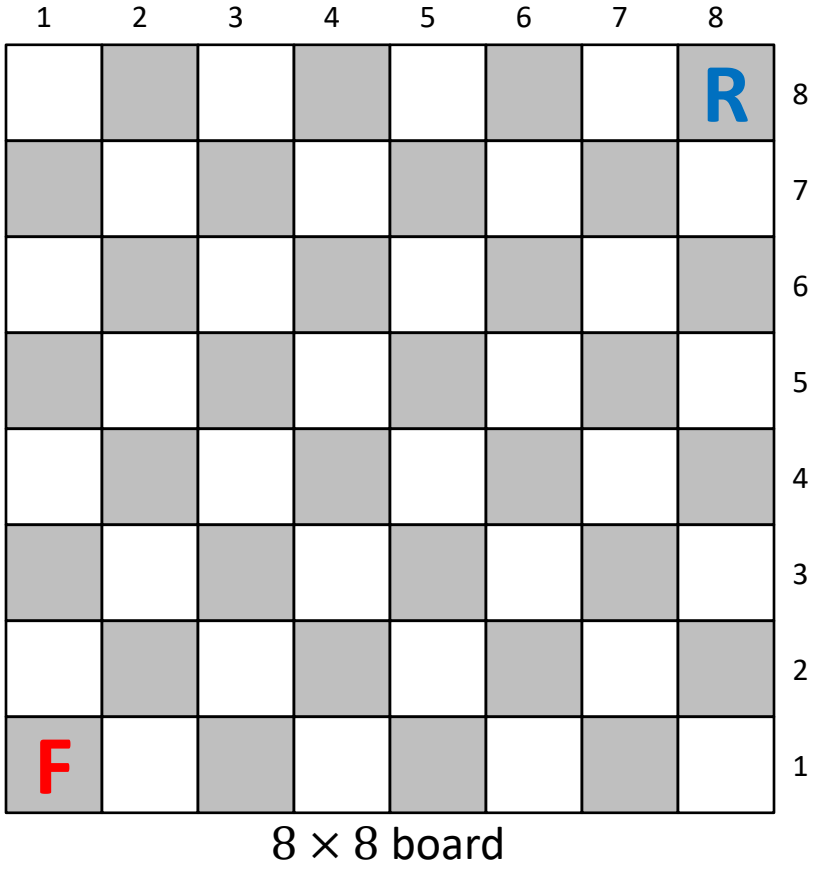
Rooster Chase

Initially, both **F** and **R** are on cells of the same color.

If **F** is to catch **R** in the next move, **R** must be in a cell horizontally or vertically adjacent to **F**'s current cell.

So, right before **F** catches **R**, they must be in cells of opposite color.

But that will never happen if **F** moves first!



8×8 board

So, if **F** moves first, **F** will never be able to catch **R**!

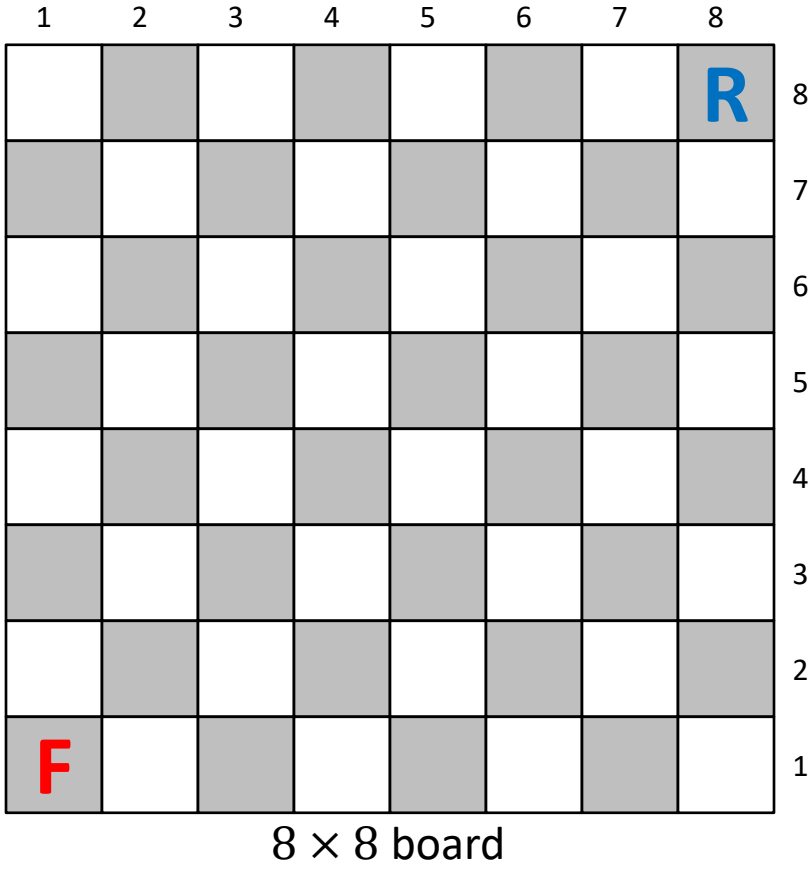
Rooster Chase

If **R** moves first, then **F** will be able to catch **R** in at most $2n - 2$ moves of its own (or $2(2n - 2)$ total moves)!

Let $(i_F, j_F) =$ **F**'s current location, and $(i_R, j_R) =$ **R**'s current location.

F's algorithm is as follows:

- $\boldsymbol{i_R - i_F > j_R - j_F}$: **F** moves up
- $\boldsymbol{i_R - i_F < j_R - j_F}$: **F** moves right
- $\boldsymbol{i_R - i_F = j_R - j_F}$: cannot happen



8×8 board

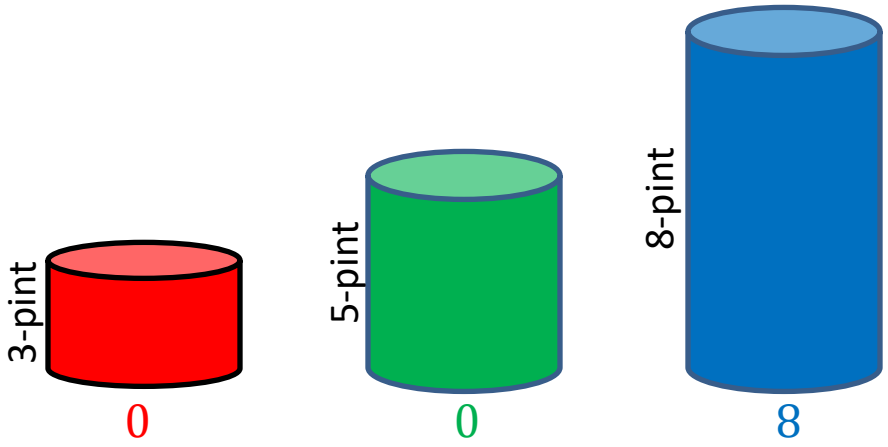
This is a *greedy algorithm* as **F** always chooses the option that looks the best (i.e., reduces the Manhattan distance to **R**) at the time of choice.

Three Jugs

Puzzle: You are given three jugs

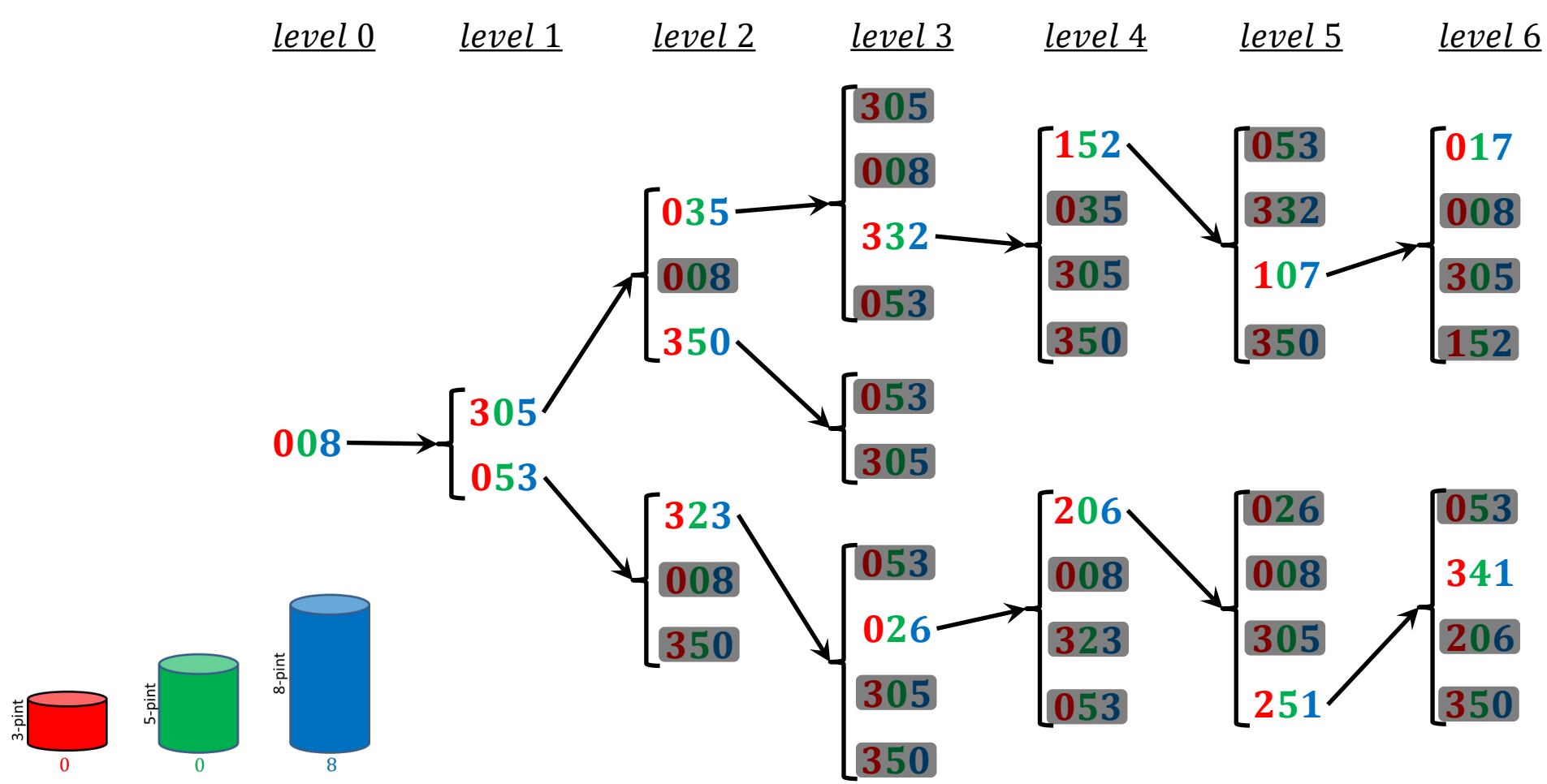
- one 8-pint jug full of water
- one empty 5-pint jug
- one empty 3-pint jug

Your task is to get exactly 4 pints of water in one of the jugs by completely filling up or emptying jugs into others. Minimize the number of times you fill up / empty jugs.

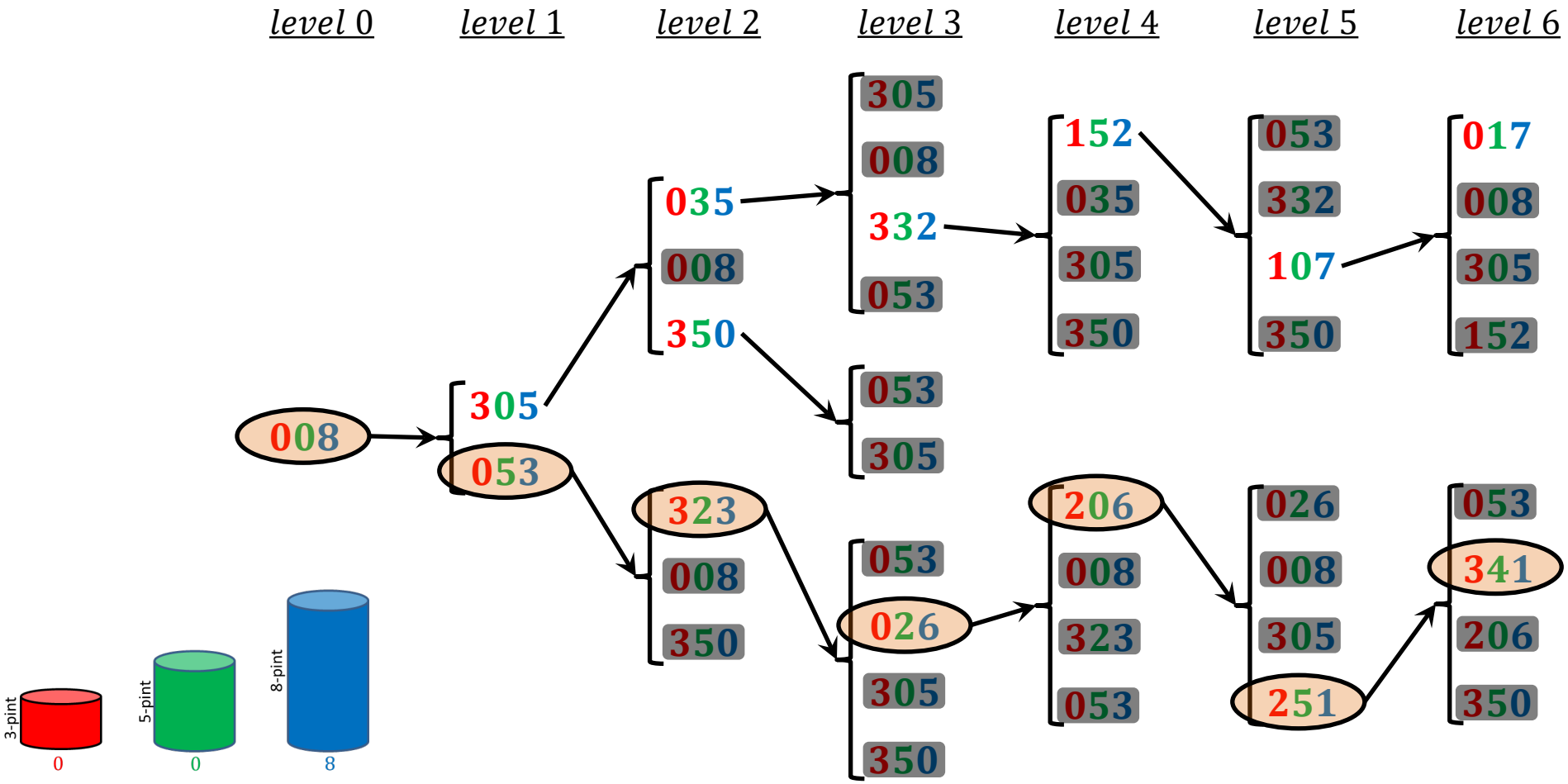


initial state

Three Jugs


level 0
level 1
level 2
level 3
level 4
level 5
level 6
008
305
053
035
008
350
323
008
350
305
008
332
053
053
305
053
026
305
350
152
035
305
350
206
008
323
053
053
332
107
350
026
008
305
251
017
008
305
152
053
341
206
350
3-pint
0
5-pint
0
8-pint
8

Three Jugs



We have just used a *state-space search algorithm* called the *breadth-first search*.